



北京航空航天大学
BEIHANG UNIVERSITY

模式识别

第七章 经典神经网络分类器

北京航空航天大学宇航学院



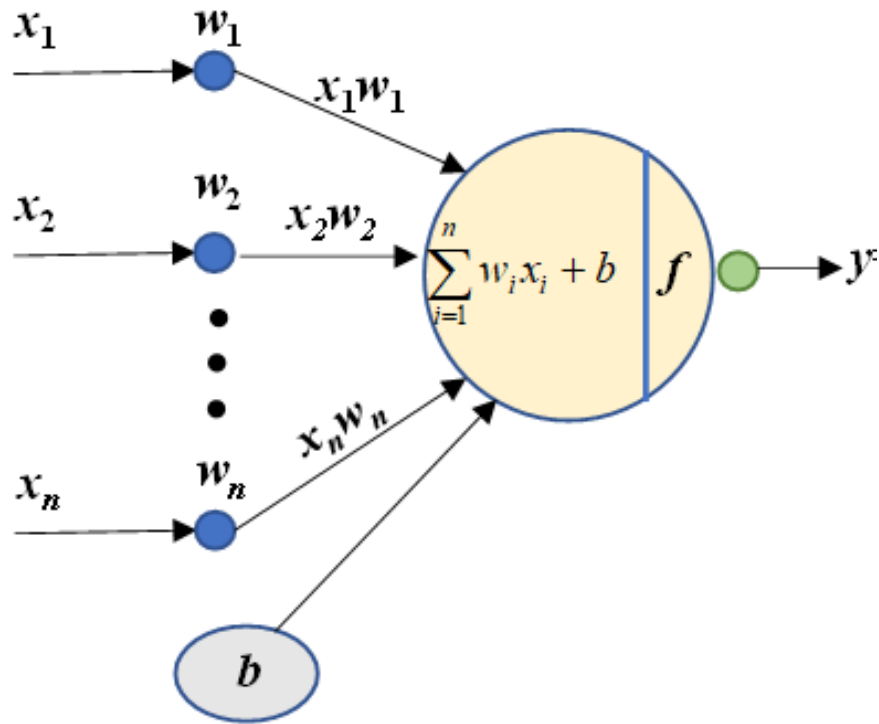
主要内容

7.1 人工神经元

7.2 感知机

7.3 BP神经网络

7.4 自组织神经网络

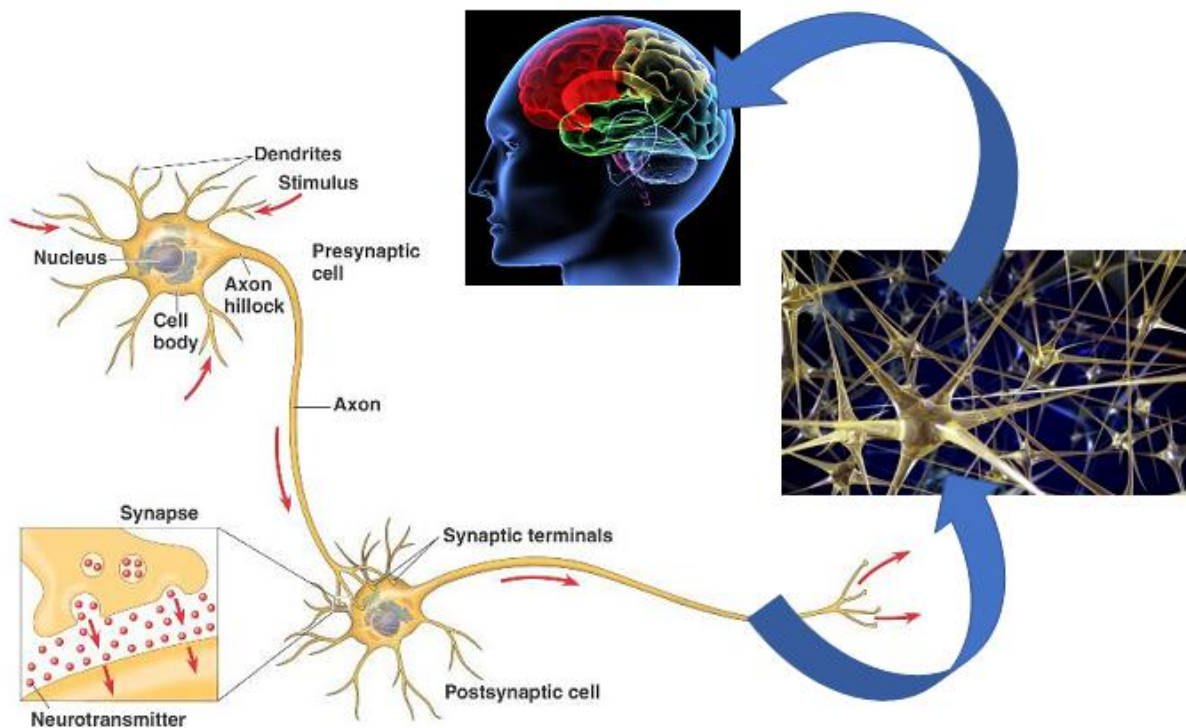




7.1 人工神经元

1、神经元组成

人类大脑有**860亿**个神经元，几乎所有的神经元都是相互联系在一起，组成复杂网络。



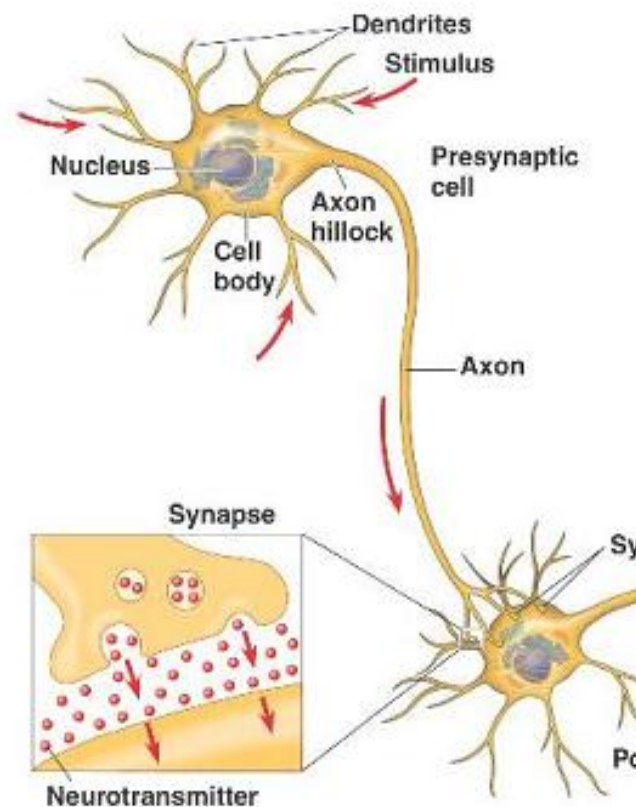


7.1 人工神经元

1、神经元组成

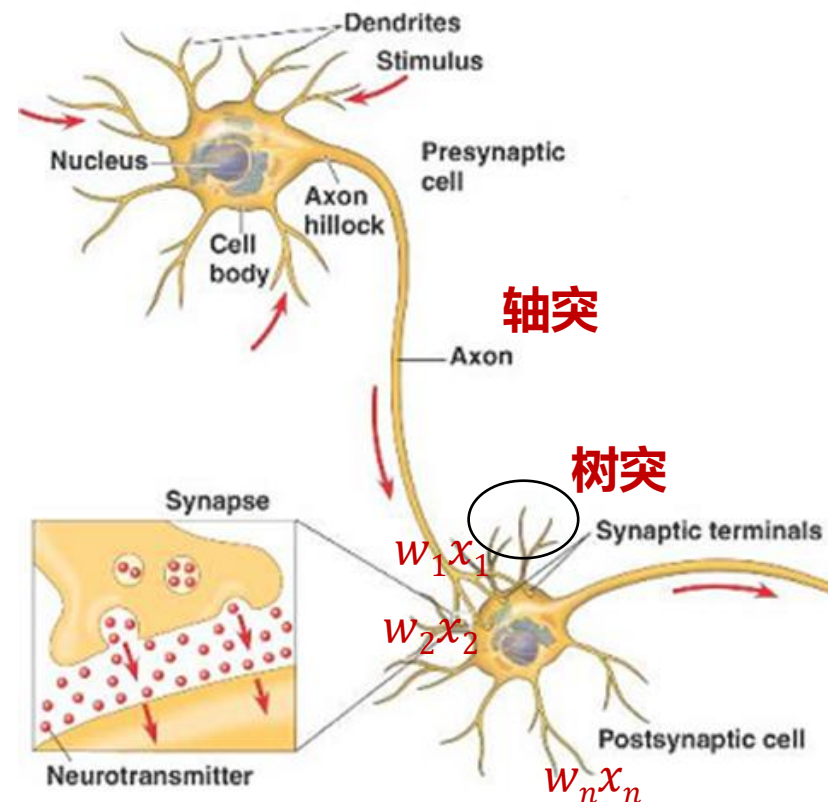
- **细胞体(Cell body):** 神经细胞的主体, 是神经细胞信息加工的主要场所, 可认为是神经元。
- **树突(Dendrites):** 细胞体外围的大量微小分支, 负责从外界获得输入信号, 这些信号在细胞体内被加工。
- **轴突(Axon):** 细胞的输出装置, 通过突触将信号 x 传输给其他神经细胞。
- **突触(Synapse):** 一个神经元的轴突与另一个神经元的树突相连接的部位。假设以 w 表示, 则传到下一个神经元树突处的信号变成了 wX 。

其中: 突触强弱(参数 w)是可学的, 它控制了一个神经元对另一个神经元影响的大小和方向(正负)。



1、神经元组成

- 树突接收到的信号 (w_1x_1)，传递到神经元(细胞体)内部，与其他树突传递过来的信号 ($w_2x_2, \dots, w_nx_n$) 进行加和： $w_1x_1 + w_2x_2 + \dots + w_nx_n$ ；
- 神经元被激活：当 $(w_1x_1 + w_2x_2 + \dots + w_nx_n) > \text{某一个固定阈值 } \theta$ ，神经元会被激活，输出信号 y ，传递给树突。

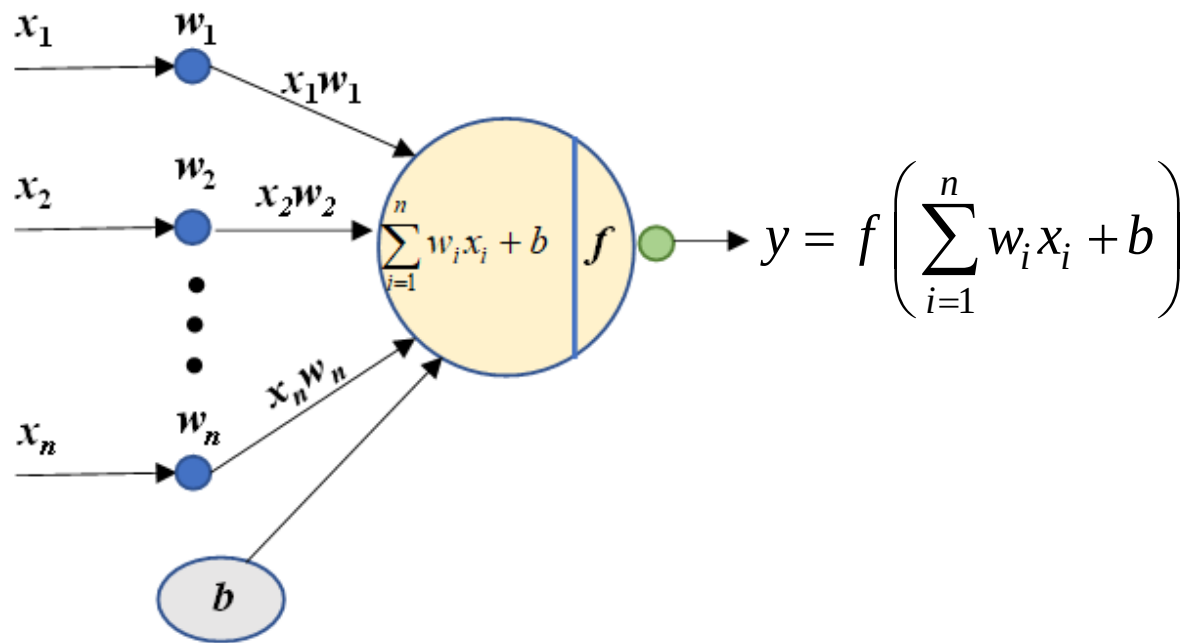


激活函数(activation function)：指的是否激活神经元的函数。在人工神经网络中，神经元节点的激活函数定义了对神经元输出的映射。



7.1 人工神经元

2、人工神经元数学模型



x_i : 来自第 i 个神经元的输入

w_i : 对应第 i 个神经元的连接权重

b : 偏置量

f : 激活函数（是否激活神经元的函数）

y : 神经元输出

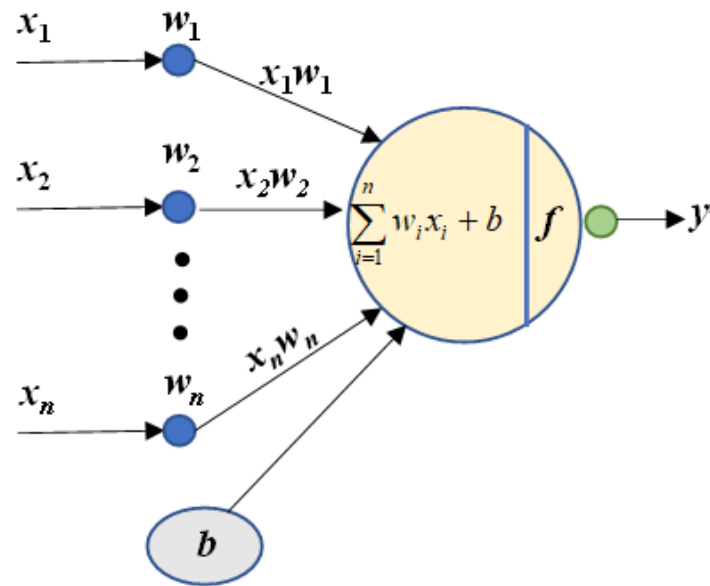


7.2 感知机

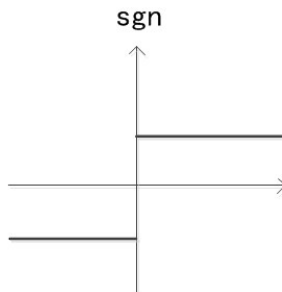
1、感知机结构

最简单的前馈式人工神经网络。

- **输入：**可接收多个输入 $(x_1, x_2, \dots, x_n | x_i \in R)$
- **输入对应权值：** $w_i \in R$
- **偏置项：** $b \in R$
- **激活函数：**通常选择**阶跃函数**作为激活函数
- **输出 y ：** $y = \text{sgn}(\sum_{i=1}^n w_i x_i + b), y \in \{+1, -1\}$



具有一层结构的感知机





7.2 感知机

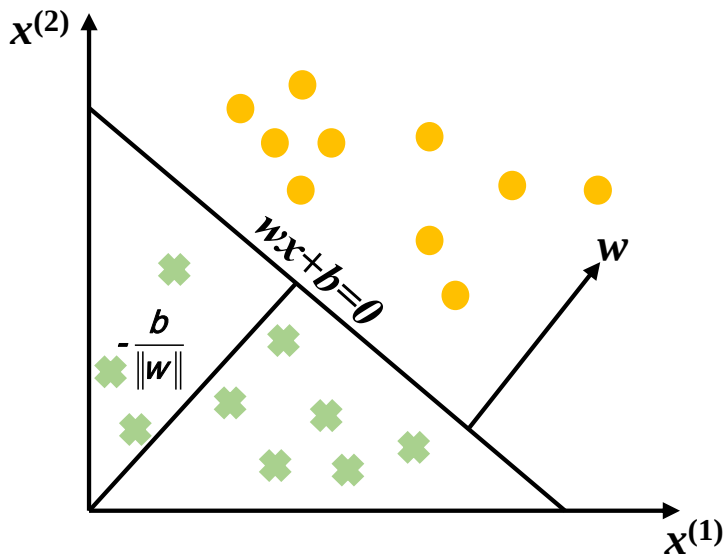
2、感知机模型求解

- 假设训练数据集线性可分，感知机学习的目标是获得一个能够将训练数据集中正、负实例完全分开的分离超平面 S 。

以二维为例：

$$y = \text{sgn}\left(\sum_{i=1}^n w_i x_i + b\right)$$

$$y \in \{+1, -1\}$$



w : 超平面的法向量;
 b : 超平面的截距。

- **感知机预测：**通过学习得到的感知机模型，对于新的输入样本，给出其对应的输出类别为1或者-1（阶跃激活函数）。



7.2 感知机

3、感知机参数学习基本原理

- 定义输入空间 R_n 中任一点 x_0 到分离超平面 S 的距离为:

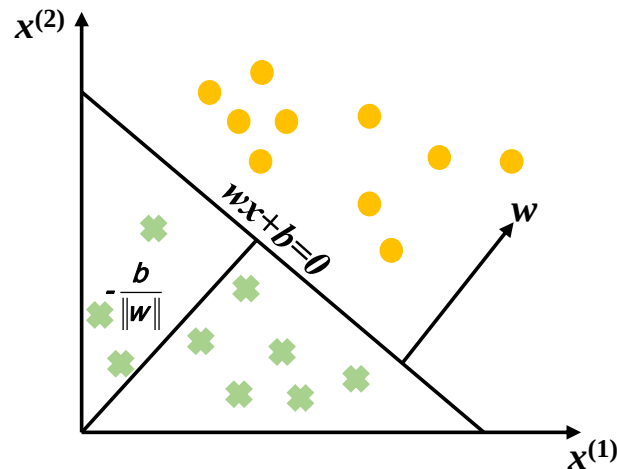
$$\frac{1}{\|w\|} |wx_0 + b| \quad \|w\|: w \text{ 的 } L_2 \text{ 范数}$$

- 对于误分类的数据 (x_i, y_i) , 如下成立:

$$-y_i(wx_i + b) \geq 0 \quad \text{为什么?}$$

误分类点 x_i 到分离超平面 S 的距离为:

$$\frac{1}{\|w\|} |wx_i + b|$$



$$y = \text{sgn}\left(\sum_{i=1}^n w_i x_i + b\right)$$
$$y \in \{+1, -1\}$$



7.2 感知机

3、感知机参数学习基本原理

- 给定 N 个样本的训练数据集: $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $y_i \in \{+1, -1\}$
- 假设分离超平面 S 的**误分类点集合为 M** , 则所有误分类点到超平面 S 的总距离为:

$$\frac{1}{\|w\|} \sum_{x_i \in M} |w^T x_i + b|$$

- 如果不考虑 $\|w\|$, 感知器学习的算法是通过学习找到一组参数 (w^*, b^*) , 使得以下损失函数 $L(w, b)$ 最小:

$$\min_{w, b} L(w, b) = - \sum_{x_i \in M} y_i (w^T x_i + b)$$

这是最优化的方法, 前一个是学习方法

误分类的数据: $-y_i (w^T x_i + b) \geq 0, y_i \in \{+1, -1\}$

- 感知器学习算法是**误分类驱动的在线学习算法**, 采用**随机梯度下降法**。任意选取一个具有参数 (w_0, b_0) 的分离超平面, 然后利用梯度下降法不断极小化损失函数。经过多次迭代, 训练得到感知器参数 (w^*, b^*)



7.2 感知机

4、感知机参数学习详细过程

算法：两类感知器参数学习算法

输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，最大迭代次数 T ，学习率 η

输出： (w^*, b^*)

随机初始化更常见

步骤 1：将参数 (w, b) 均初始化为 0；

步骤 2：训练数据集中选择数据 (x_i, y_i) ；

如果 $y_i(w x_i + b) \leq 0$ ，则 (x_i, y_i) 为误分类样本。利用误分类样本，进行参数更新：

梯度下降法

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

步骤 3：重复步骤 2，直到达到最大迭代次数 T 。

$$\min_{w, b} L(w, b) = - \sum_{x_i \in M} y_i (w x_i + b)$$

误分类的数据：

$$-y_i(w x_i + b) \geq 0$$

η ：学习步长，也称为学习率。
用于控制每一步调整权值的幅度

对 w 的偏导数： $-y_i x_i$

对 b 的偏导数： $-y_i$



7.2 感知机

5、感知机算例

- 某个训练数据集，其正实例点为：

$$x_1 = [3 \ 3]^T, y_1 = 1$$

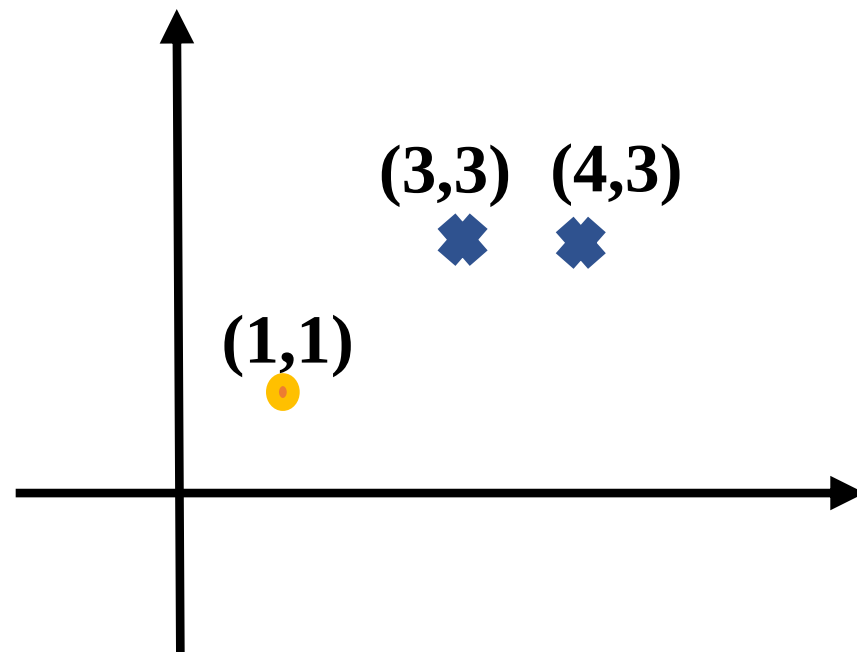
$$x_2 = [4 \ 3]^T, y_2 = 1$$

- 负实例点为：

$$x_3 = [1 \ 1]^T, y_3 = -1$$

- 试用感知机学习算法求感知机模型：

$$g(x) = f(wx + b), w = [w^{(1)} \ w^{(2)}]^T$$





7.2 感知机

5、感知机算例

➤ 构建最优化问题: $\min_{w,b} L(w,b) = -\sum_{x_i \in M} y_i (w^T x_i + b)$

➤ 根据感知机学习算法求解 w, b 。设学习率 $\eta = 1$

① 给出权值的初始值 $w_0 = [0 \ 0]^T, b_0 = 0$

② 对于 $x_1 = [3 \ 3]^T$, 得到 $y_1(w_0 x_1 + b_0) = 0$ 。则判定 x_1 未被正确分类, 利用该点更新 w, b

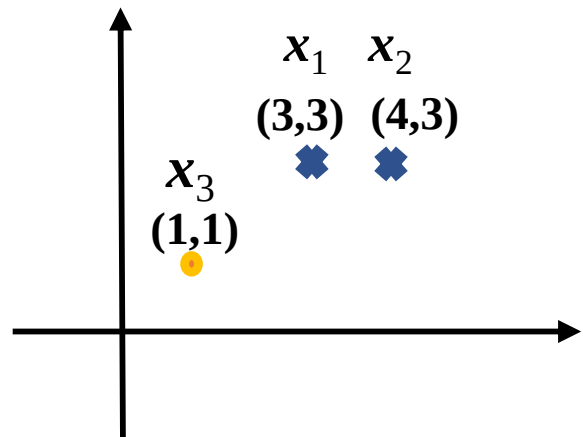
$$w_1 = w_0 + y_1 x_1 = [3 \ 3]^T$$

$$b_1 = b_0 + y_1 = 1$$

$$\begin{aligned} w &\leftarrow w + \eta y_i x_i \\ b &\leftarrow b + \eta y_i \end{aligned}$$

得到线性模型:

$$w_1 x + b_1 = 3x^{(1)} + 3x^{(2)} + 1$$





7.2 感知机

5、感知机算例

$$\mathbf{w}_1^T \mathbf{x} + b_1 = 3x^{(1)} + 3x^{(2)} + 1$$



- ③ 对于 x_1 和 x_2 , 得到 $y_1(\mathbf{w}_1^T \mathbf{x}_1 + b_1) > 0, y_2(\mathbf{w}_2^T \mathbf{x}_2 + b_2) > 0$ 。 则判定 x_1 和 x_2 被正确分类, 此时不更新 w, b
- ④ 对于 $x_3 = [1 \ 1]^T$, 得到 $y_3(\mathbf{w}_1^T \mathbf{x}_3 + b_1) < 0$ 。 则判定 x_3 被错误分类, 利用该点更新 w, b

$$\mathbf{w}_2 = \mathbf{w}_1 + y_3 \mathbf{x}_3 = [2 \ 2]^T, b_2 = b_1 + y_3 = 0$$

得到线性模型: $\mathbf{w}_2^T \mathbf{x} + b_2 = 2x^{(1)} + 2x^{(2)}$

- ⑤ 不断输入实例点, 判定该实例点是否为误分类点, 若是, 则利用该点更新权值, 直至:

$$\mathbf{w}_7 = [1 \ 1]^T, b_7 = -3$$

对所有数据点 $y_i(\mathbf{w}_7^T \mathbf{x}_i + b_7) > 0$, 没有误分类点, 损失函数达到极小。

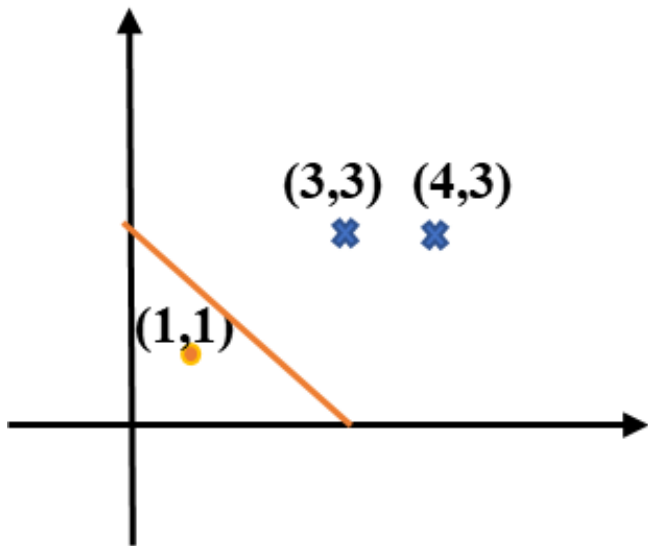


7.2 感知机

5、感知机算例

此时得到的分离超平面为： $x^{(1)} + x^{(2)} - 3 = 0$

感知机模型为： $g(x) = f(x^{(1)} + x^{(2)} - 3)$



迭代次数	误分类点	$wx + b$
0		0
1	x_1	$3x^{(1)}+3x^{(2)}+1$
2	x_3	$2x^{(1)}+2x^{(2)}$
3	x_3	$x^{(1)}+x^{(2)}-1$
4	x_3	$x^{(2)}-2$
5	x_1	$3x^{(1)}+3x^{(2)}-1$
6	x_3	$2x^{(1)}+2x^{(2)}-2$
7	x_3	$x^{(1)}+x^{(2)}-3$
8	0	$x^{(1)}+x^{(2)}-3$



7.2 感知机

5、感知机算例

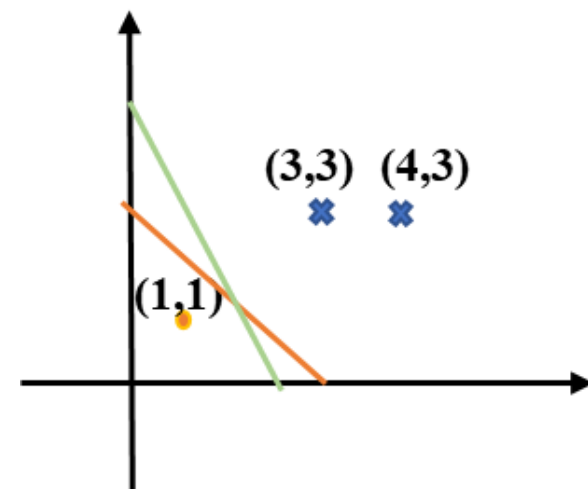
讨论：

- 在计算中误分类点先后取 $x_1, x_3, x_3, x_3, x_1, x_3, x_3$ ，分离超平面（红色实线）为：

$$x^{(1)} + x^{(2)} - 3 = 0$$

- 在计算中误分类点先后取 $x_1, x_3, x_3, x_3, x_2, x_3, x_3, x_3, x_1, x_3, x_3$ ，分离超平面（绿色实线）为：

$$2x^{(1)} + x^{(2)} - 5 = 0$$



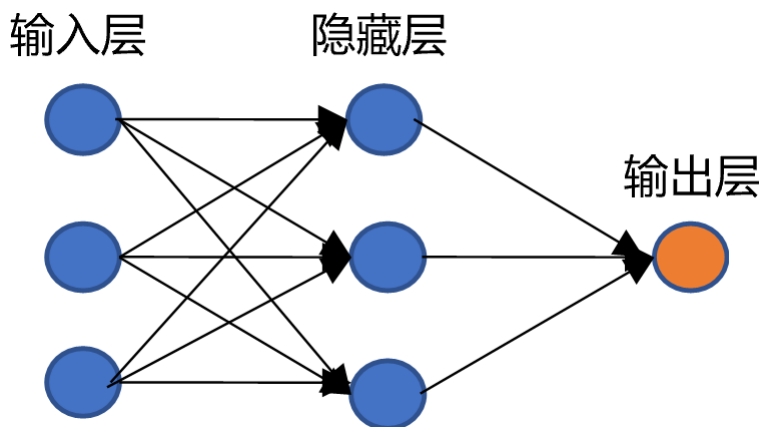


7.2 感知机

6、多层感知机

2-3层

- **单层感知机**可以很好的**解决线性分类**问题，不适用于解决非线性问题，例如简单的异或问题。
- 通过将多个神经元分层组合，即构建**多层感知机**,可以很好的解决非线性可分问题，通常将多层感知机称之为**神经网络**。
- 20世纪60年代，学者发现**感知机学习算法不能直接应用到这种多层感知机模型参数学习中**。



多个感知机组成的两层结构(层与层之间全连接)



7.2 感知机

6、多层感知机

➤ 原因解释

感知机学习算法以训练样本被错分的程度作为目标函数，训练过程中每次出现错分，就对权值进行修改，使其朝着目标函数相对于权值的负梯度方向更新，直到目标函数取得极小值，此时没有训练样本被错分。

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = - \sum_{\mathbf{x}_i \in M} y_i \mathbf{x}_i \quad \nabla_b L(\mathbf{w}, b) = - \sum_{\mathbf{x}_i \in M} y_i$$



$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad b \leftarrow b + \eta y_i$$



$$\min_{\mathbf{w}, b} L(\mathbf{w}, b) = - \sum_{\mathbf{x}_i \in M} y_i (\mathbf{w}^T \mathbf{x}_i + b)$$

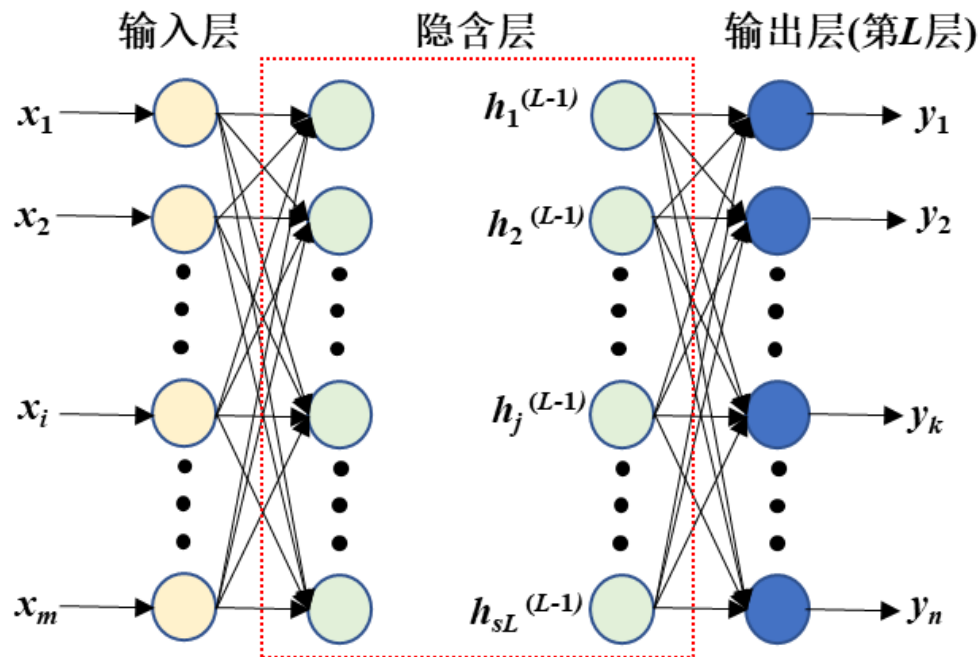
感知机学习算法是一种单层网络的学习算法。在多层网络中，它只能改变最后层的权值（通过判断训练样本被错分的情况，代入如上权值更新公式进行权值更新）。此外，感知机采用了阶跃激活函数（其导数为0），误差无法反向传播。



7.3 BP网络

1、BP网络结构

- **BP算法：误差反向传播算法** (Error Back Propagation Training) , 系统解决了多层神经网络隐含层神经元连接权值学习的问题。
- **BP网络：基于BP算法进行误差校正的多层前馈网络**。用Sigmoid激活函数替代了阶跃函数。



信号前向传播

误差反向传播



7.3 BP网络

1、BP网络结构

➤ Sigmoid激活函数

① 具有指数函数形式:

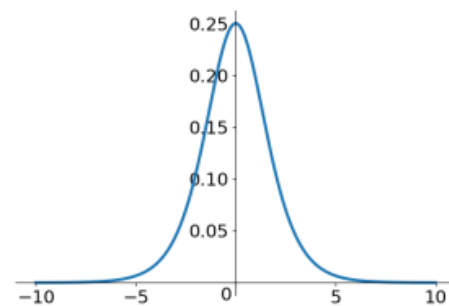
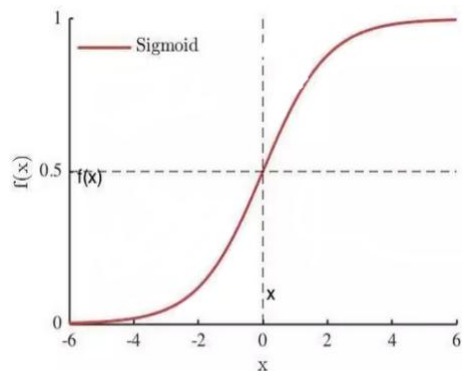
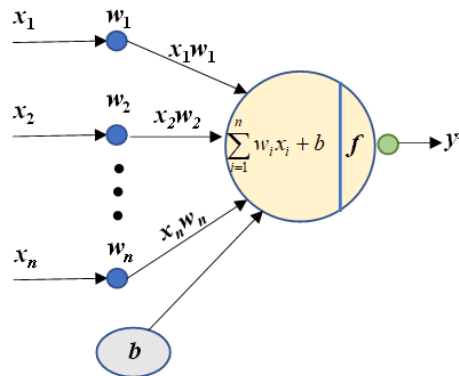
$$f(x) = \frac{1}{1 + e^{-x}}$$

将一个实数输入映射到[0,1]范围。

② 该激活函数导数为:

$$\begin{aligned} f'(x) &= \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}}\right) \\ &= f(x)(1 - f(x)) \end{aligned}$$

在定义域内处处可导，且两侧导数逐渐趋近于0





7.3 BP网络

2、BP网络参数设置

➤ 输入向量为:

$$\mathbf{x} = [x_1 \ x_2 \ \dots \ x_i \ \dots \ x_m], i = 1, 2, \dots, m$$

➤ 输出向量为:

$$\mathbf{y} = [y_1 \ y_2 \ \dots \ y_k \ \dots \ y_n], k = 1, 2, \dots, n$$

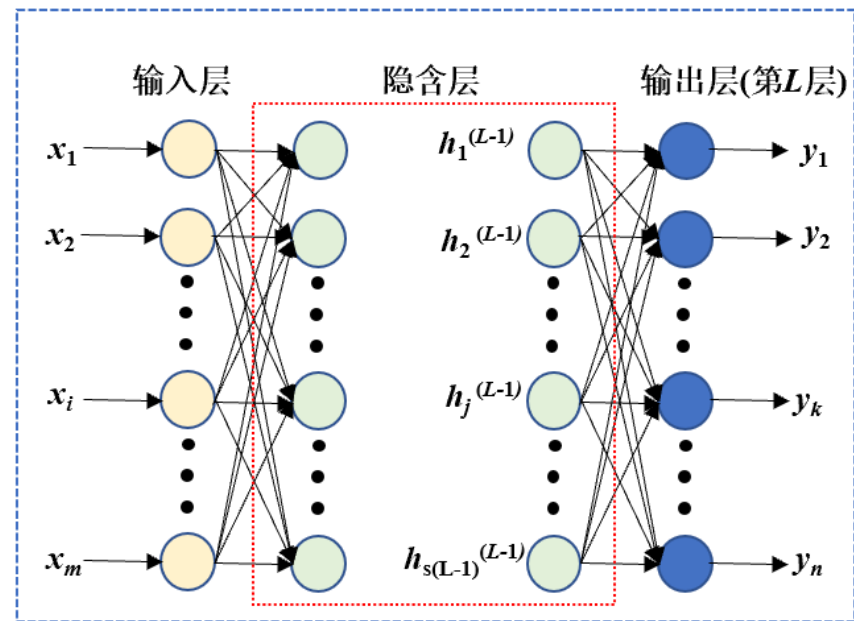
➤ 第 l 隐含层各神经元的输出为:

$$\mathbf{h}^{(l)} = [h_1^{(l)} \ h_2^{(l)} \ \dots \ h_j^{(l)} \ \dots \ h_{s_l}^{(l)}], j = 1, 2, \dots, s_l$$



$$h_i^{(l)} \quad \boxed{?}$$

s_l : 第 l 层神经元的个数





7.3 BP网络

2、BP网络参数设置

➤ 第 l 隐含层各神经元的输出:

$$h_i^{(l)} = f\left(net_i^{(l)}\right)$$

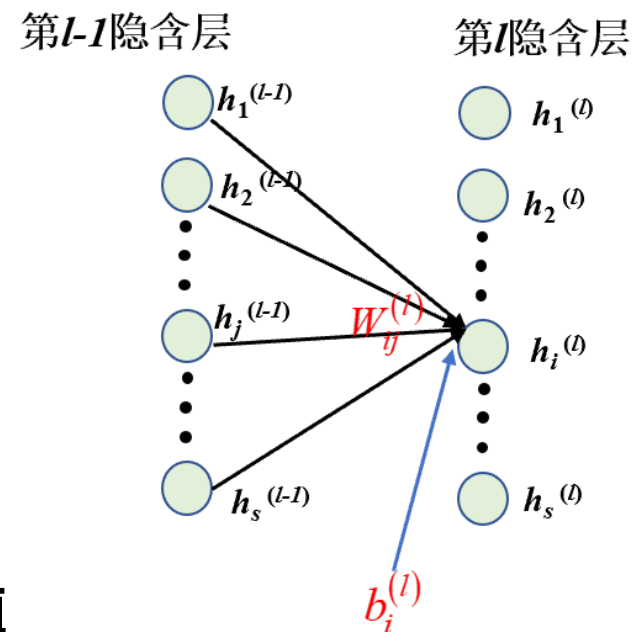
$$net_i^{(l)} = \sum_{j=1}^{s_{l-1}} W_{ij}^{(l)} h_j^{(l-1)} + b_i^{(l)}$$

$net_i^{(l)}$: 第 l 层第 i 个神经元的输入

$f()$: 神经元激活函数, BP网络通常采用Sigmoid激活函数

$W_{ij}^{(l)}$: 第 $l-1$ 层第 j 个神经元与第 l 层第 i 个神经元之间的连接权值

$b_i^{(l)}$: 第 l 层第 i 个神经元的偏置





7.3 BP网络

3、BP推导

假设有 M 个训练样本 $\{(x(1), d(1)), (x(2), d(2)), \dots, (x(M), d(M))\}$, $d(i)$ 是样本 $x(i)$ 的期望输出

➤ **BP算法**: 通过最优化各层神经元的输入权值以及偏置量, 使得BP网络输出尽可能接近期望输出。

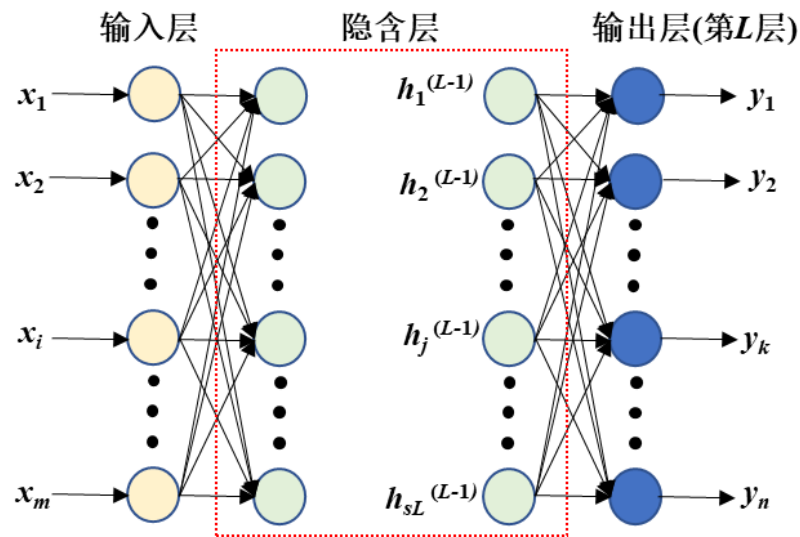
把 δw 和 δb 求出来, 但是不更新 w 和 b , 只到一次更新遍历完全部的样本, 再对 δw 和 δb 求和取平均更新 (减)

采用**批量样本更新算法**, 即对于给定的 M 个训练样本, 定义**误差函数**为所有样本的误差均值:

$$E = \frac{1}{M} \sum_{i=1}^M E(i)$$

$E(i)$ 为样本 $x(i)$ 的训练误差, 采用均方误差损失函数:

$$E(i) = \frac{1}{2} \sum_{k=1}^n (d_k(i) - y_k(i))^2, n \text{ 代表输出的维度}$$





7.3 BP网络

3、BP推导

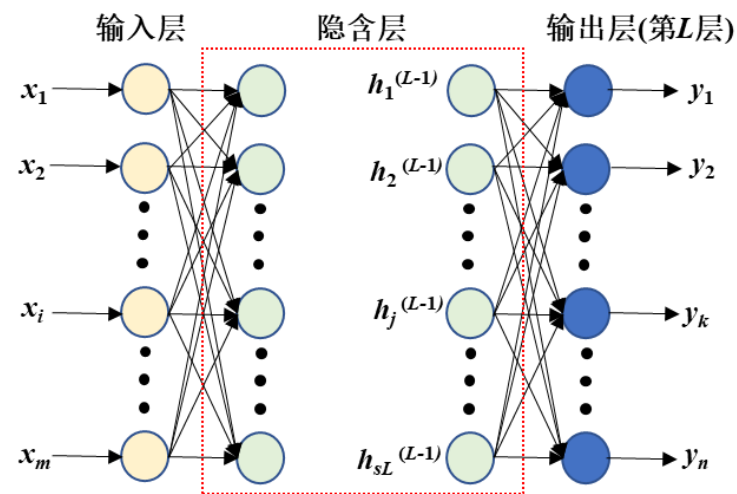
➤ 将 $E(i)$ 代入 E 得到给定 M 个样本的误差均值:

$$E = \frac{1}{2M} \sum_{i=1}^M \sum_{k=1}^n (d_k(i) - y_k(i))^2$$

➤ 在采用BP算法每次迭代时, 按照如下方式对权值和偏置量进行更新:

$$W_{ij}^{(l)} \leftarrow W_{ij}^{(l)} - \eta \frac{\partial E}{\partial W_{ij}^{(l)}} \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial E}{\partial b_i^{(l)}}$$

η : 为学习率, 取值范围(0,1)



BP网络权值和偏置更新的关键是计算误差损失函数关于权值和偏置的偏导数。

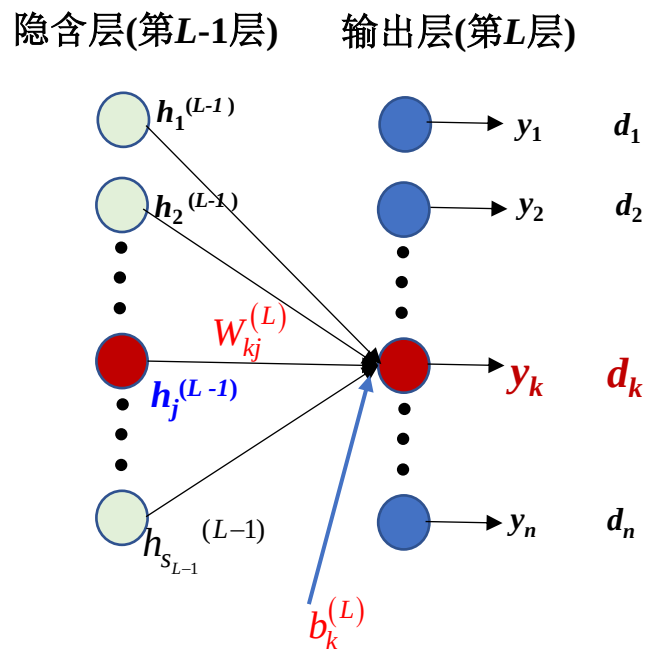


7.3 BP网络

3、BP推导

➤ 对于单个训练样本 $x(i)$ ，隐含层-》输出层 $E(i)$ 关于权值的偏导数为：

$$\begin{aligned}\frac{\partial E(i)}{\partial W_{kj}^{(L)}} &= \frac{\partial}{\partial W_{kj}^{(L)}} \left(\frac{1}{2} \sum_{k_1=1}^n (d_{k_1}(i) - y_{k_1}(i))^2 \right) \\ &= \frac{\partial}{\partial W_{kj}^{(L)}} \left[\frac{1}{2} (d_k(i) - y_k(i))^2 \right] \quad \text{仅与输出层第 } k \text{ 个神经元有关} \\ &= -(d_k(i) - y_k(i)) \frac{\partial y_k(i)}{\partial W_{kj}^{(L)}} \\ &= -(d_k(i) - y_k(i)) \frac{\partial y_k(i)}{\partial net_k^{(L)}} \frac{\partial net_k^{(L)}}{\partial W_{kj}^{(L)}} \quad \text{链式法则} \\ &= -(d_k(i) - y_k(i)) \underbrace{f(x)' \Big|_{x=net_k^{(L)}}}_{\text{链式法则}} \frac{\partial net_k^{(L)}}{\partial W_{kj}^{(L)}} \\ &= -(d_k(i) - y_k(i)) f(x)' \Big|_{x=net_k^{(L)}} h_j^{(L-1)}\end{aligned}$$



$$y_k^{(L)} = f\left(net_k^{(L)}\right)$$
$$net_k^{(L)} = \sum_{j=1}^{s_{L-1}} W_{kj}^{(L)} h_j^{(L-1)} + b_k^{(L)}$$



7.3 BP网络

3、BP推导

➤ 同理，对于单个训练样本 $x(i)$ ，隐含层-》输出层 $E(i)$ 关于偏置量的偏导数为：

$$\frac{\partial E(i)}{\partial b_k^{(L)}} = -(d_k(i) - y_k(i))f'(x)|_{x = ne_{t_k^{(L)}}}$$

令 $\delta_k^{(L)} = -(d_k(i) - y_k(i))f'(x)|_{x = ne_{t_k^{(L)}}}$

若 $f'(x) = 1$ ，则 $\delta_k^{(L)} = -(d_k(i) - y_k(i))$ 为网络输出与期望输出之差。隐含层-》输出层 $E(i)$ 关于权值和偏置量的偏导数可简化为：

$$\frac{\partial E(i)}{\partial W_{kj}^{(L)}} = \delta_k^{(L)} h_j^{(L-1)} \quad \frac{\partial E(i)}{\partial b_k^{(L)}} = \delta_k^{(L)}$$



7.3 BP网络

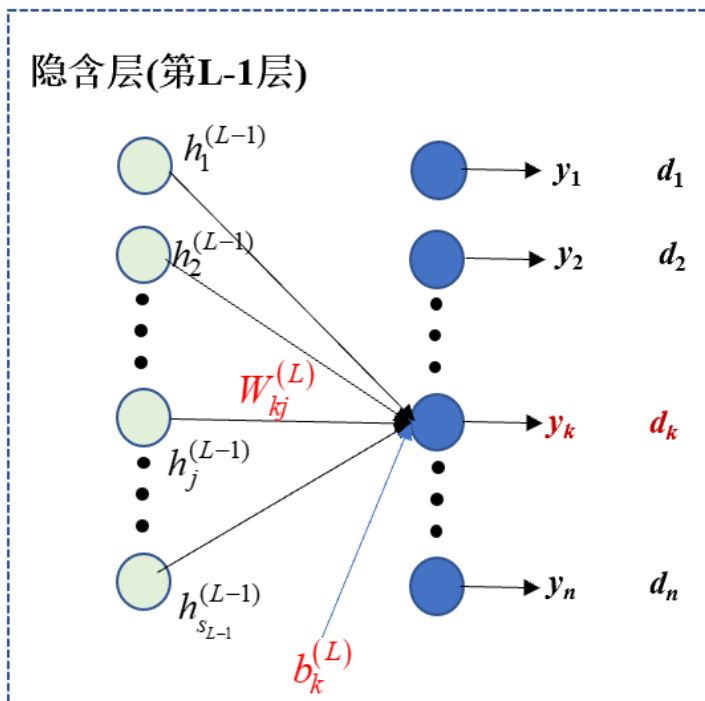
3、BP推导

➤ 对于单个训练样本 $x(i)$ ，隐含层-》输出层 $E(i)$ 关于权值和偏置量的偏导数的直观解释：

$$\frac{\partial E(i)}{\partial W_{kj}^{(L)}} = \underbrace{\delta_k^{(L)}}_{\text{误差}} \underbrace{h_j^{(L-1)}}_{\text{输入}}$$

$$\frac{\partial E(i)}{\partial b_k^{(L)}} = \delta_k^{(L)}$$

误差反向传播



- 输出层（第 L 层）中第 k 个神经元关于与其连接的第 $L-1$ 隐含层中第 j 个神经元的权值的偏导数为：**误差***第 $L-1$ 隐含层中第 j 个神经元到第 L 层输出层中第 k 个神经元的**输入**；
- 而关于偏置量的偏导数=第 L 层第 k 个神经元误差。



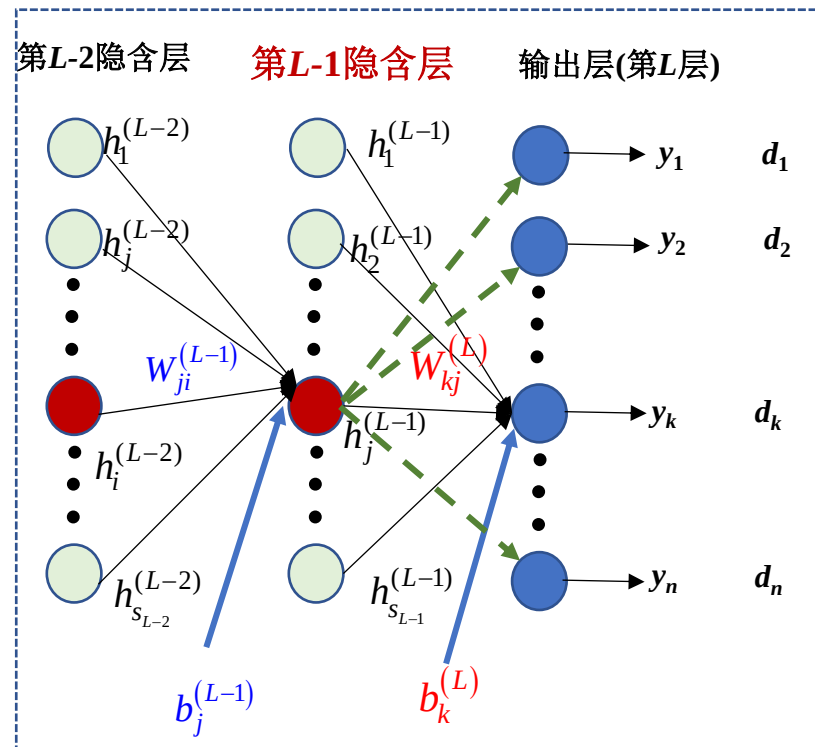
7.3 BP网络

3、BP推导

➤ 隐含层 $L-1$ 层, $E(i)$ 关于权值的偏导数为:

$$\begin{aligned}
 \frac{\partial E(i)}{\partial W_{ji}^{(L-1)}} &= \frac{\partial}{\partial W_{ji}^{(L-1)}} \left(\frac{1}{2} \sum_{k=1}^n (d_k(i) - y_k(i))^2 \right) \\
 &= - \sum_{k=1}^n (d_k(i) - y_k(i)) \frac{\partial y_k(i)}{\partial W_{ji}^{(L-1)}} \quad \text{链式法则} \\
 &= - \sum_{k=1}^n (d_k(i) - y_k(i)) \frac{\partial f(net_k^L)}{\partial W_{ji}^{(L-1)}} \quad \text{链式法则} \\
 &= - \sum_{k=1}^n (d_k(i) - y_k(i)) \frac{\partial f(net_k^L)}{\partial net_k^{(L)}} \frac{\partial net_k^{(L)}}{\partial W_{ji}^{(L-1)}} \\
 &= - \sum_{k=1}^n (d_k(i) - y_k(i)) f(x)' \Big|_{x=net_k^{(L)}} \frac{\partial net_k^{(L)}}{\partial W_{ji}^{(L-1)}}
 \end{aligned}$$

结合
定义



$$\begin{aligned}
 net_k^{(L)} &= \sum_{j=1}^{s_{L-1}} W_{kj}^{(L)} h_j^{(L-1)} + b_k^{(L)} \\
 &= \sum_{j=1}^{s_{L-1}} W_{kj}^{(L)} f \left(\sum_{i=1}^{s_{L-2}} W_{ji}^{(L-1)} h_i^{(L-2)} + b_j^{(L-1)} \right) + b_k^{(L)} \\
 &= \sum_{j=1}^{s_{L-1}} W_{kj}^{(L)} f \left(net_j^{(L-1)} \right) + b_k^{(L)}
 \end{aligned}$$

进一步得到





7.3 BP网络

输入 $h*w$ ：整个输入net

整个输入-输出 $h/f(\text{net})$ ：激活函数

下一层的输出 h =下一层的输入 h

3、BP推导

➤ 隐含层 $L-1$ 层， $E(i)$ 关于权值的偏导数进一步表示为：

$$\begin{aligned} \frac{\partial E(i)}{\partial W_{ji}^{(L-1)}} &= -\sum_{k=1}^n (d_k(i) - y_k(i)) f(x)' \Big|_{x=\text{net}_k^{(L)}} \frac{\partial \text{net}_k^{(L)}}{\partial W_{ji}^{(L-1)}} \quad \text{链式法则} \quad \text{net}_k^{(L)} = \sum_{j=1}^{s_{L-1}} W_{kj}^{(L)} f(\text{net}_j^{(L-1)}) + b_k^{(L)} \\ &= -\sum_{k=1}^n (d_k(i) - y_k(i)) f(x)' \Big|_{x=\text{net}_k^{(L)}} \frac{\partial \text{net}_k^{(L)}}{\partial f(\text{net}_j^{(L-1)})} \frac{\partial f(\text{net}_j^{(L-1)})}{\partial \text{net}_j^{(L-1)}} \frac{\partial \text{net}_j^{(L-1)}}{\partial W_{ji}^{(L-1)}} \\ &= -\sum_{k=1}^n (d_k(i) - y_k(i)) f(x)' \Big|_{x=\text{net}_k^{(L)}} W_{kj}^{(L)} f(x)' \Big|_{x=\text{net}_j^{(L-1)}} h_i^{(L-2)} \end{aligned}$$

其中：

$$\frac{\partial \text{net}_k^{(L)}}{\partial f(\text{net}_j^{(L-1)})} = W_{kj}^{(L)}$$

$$\frac{\partial f(\text{net}_j^{(L-1)})}{\partial \text{net}_j^{(L-1)}} = f(x)' \Big|_{x=\text{net}_j^{(L-1)}}$$

$$\frac{\partial \text{net}_j^{(L-1)}}{\partial W_{ji}^{(L-1)}} = h_i^{(L-2)}$$

结合
定义

$$\text{net}_j^{(L-1)} = \sum_{i=1}^{s_{L-2}} W_{ji}^{(L-1)} h_i^{(L-2)} + b_j^{(L-1)}$$



7.3 BP网络

3、BP推导

➤ 同理，得到隐含层 $L-1$ 层， $E(i)$ 关于偏置的偏导数：

$$\frac{\partial E(i)}{\partial b_j^{(L-1)}} = - \sum_{k=1}^n (d_k(i) - y_k(i)) f'(x) \Big|_{x=net_k^{(L)}} W_{kj}^{(L)} f'(x) \Big|_{x=net_j^{(L-1)}}$$

令：

第 $L-1$ 层的误差

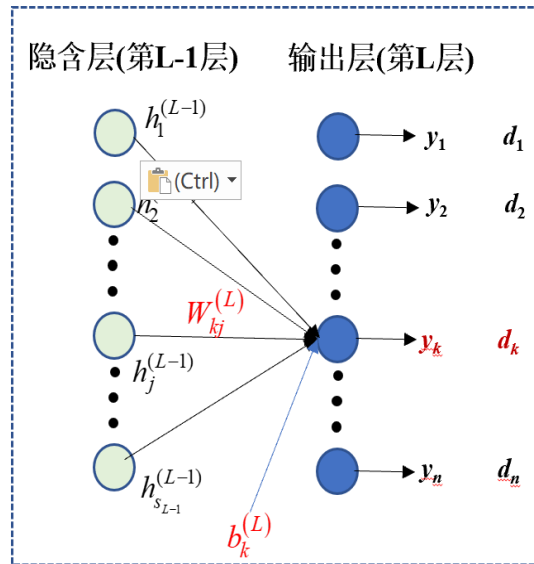
$$\begin{aligned} \delta_j^{(L-1)} &= - \sum_{k=1}^n (d_k(i) - y_k(i)) f'(x) \Big|_{x=net_k^{(L)}} W_{kj}^{(L)} f'(x) \Big|_{x=net_j^{(L-1)}} \\ &= \sum_{k=1}^n \delta_k^{(L)} W_{kj}^{(L)} f'(x) \Big|_{x=net_j^{(L-1)}} \end{aligned}$$

第 L 层的误差

= 上一层（输出层即第 L 层）所有神经元对应的误差 * 上一层权重 $W_{kj}^{(L)}$ * 激活函数导数的求和

➤ 隐含层 $L-1$ 层， $E(i)$ 关于权值和偏置的偏导数可进一步简化为：

$$\frac{\partial E(i)}{\partial W_{ji}^{(L-1)}} = \delta_j^{(L-1)} h_i^{(L-2)} \quad \frac{\partial E(i)}{\partial b_j^{(L-1)}} = \delta_j^{(L-1)}$$





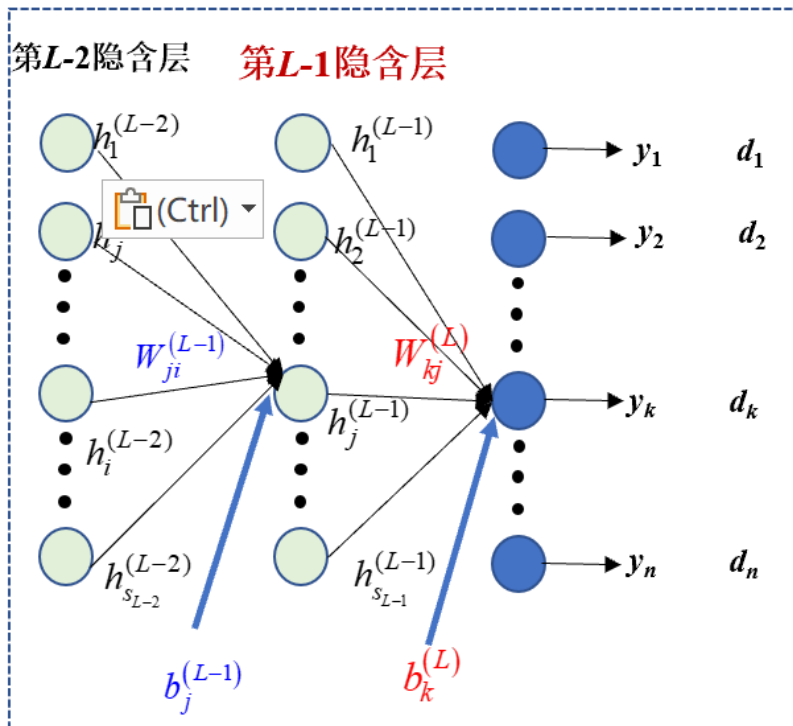
7.3 BP网络

3、BP推导

➤ 隐含层 $L - 1$ 层, $E(i)$ 关于权值和偏置的偏导数的直观解释:

$$\frac{\partial E(i)}{\partial W_{ji}^{(L-1)}} = \delta_j^{(L-1)} h_i^{(L-2)} \frac{\partial E(i)}{\partial b_j^{(L-1)}} = \delta_j^{(L-1)}$$

误差反向传播



- 第 $L - 1$ 隐含层（当前层）中第 j 个神经元关于与其连接的第 $L - 2$ 隐含层（上一层）中第 i 个神经元的权值的偏导数为：第 $L - 1$ 层第 j 个神经元误差（由第 L 层输出层反传回来的）* 第 $L-2$ 隐含层中第 i 个神经元到第 $L-1$ 隐含层中第 j 个神经元的输入 $(h_i^{(L-2)})$ ；
- 第 $L-1$ 层的误差是第 L 层误差的反向传播。



7.3 BP网络

3、BP推导

➤ 依次类推，得到第 l 层 ($2 \leq l \leq L - 1$)， $E(i)$ 关于权值和偏置的偏导数：

$$\frac{\partial E(i)}{\partial W_{ji}^{(l)}} = \delta_j^{(l)} h_i^{(l-1)} \quad \frac{\partial E(i)}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

误差反向传播

其中， $\delta_j^{(l)} = \sum_{k=1}^{s_{l+1}} W_{kj}^{(l+1)} \delta_k^{(l+1)} f(x)' \Big|_{x = ne_{t_j}^{(l)}}$

输出误差通过隐含层向输入层逐层反向传播，并将误差分摊给各层所有节点（获得所有层的误差估计），按误差函数的负梯度方向修改各节点连接权重。



7.3 BP网络

4、BP具体实现步骤

若采用批量样本更新算法对神经网络的权值和偏置进行更新，**则BP算法**

步骤为：

步骤1：输入 M 个样本：

$x = [x_1 \quad x_2 \quad \dots \quad x_i \quad \dots \quad x_m], i = 1, 2, \dots, m$ 每个样本为 m 维向量

$y = [y_1 \quad y_2 \quad \dots \quad y_k \quad \dots \quad y_n], k = 1, 2, \dots, n$ 标签为 n 维向量

步骤2：网络初始化。初始化各连接权值，给定学习率。

对所有层 $2 \leq l \leq L$ ：

$$\Delta W^{(l)} = 0, \Delta b^{(l)} = 0$$

对所有层，初始化权重和偏置更新量，开始时均为0；



7.3 BP网络

4、BP具体实现步骤

步骤3：遍历所有样本训练执行如下操作：

- a) 对样本 $x(i)$ ，计算误差损失函数 $E(i)$
- b) 利用BP算法，计算每个样本误差损失函数 $E(i)$ 关于各层权值和偏置的偏导数

$$\frac{\partial E(i)}{\partial W_{ji}^{(l)}} \quad \frac{\partial E(i)}{\partial b_j^{(l)}}$$

得到梯度矩阵 $\nabla W^{(l)}(i)$ 和向量 $\nabla b^{(l)}(i)$ ；

- c) 对所有样本的梯度矩阵 $\nabla W^{(l)}(i)$ 和向量 $\nabla b^{(l)}(i)$ 进行求和：

$$\Delta W^{(l)} = \Delta W^{(l)} + \nabla W^{(l)}(i) \quad \Delta b^{(l)} = \Delta b^{(l)} + \nabla b^{(l)}(i)$$



7.3 BP网络

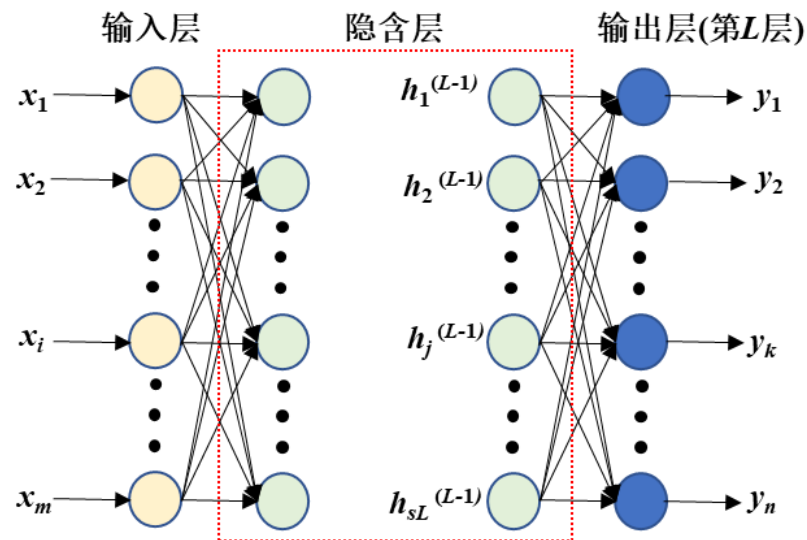
4、BP具体实现步骤

步骤4：进行权值和偏置的更新

$$W^{(l)} = W^{(l)} - \eta \frac{1}{M} \Delta W^{(l)} \quad b^{(l)} = b^{(l)} - \eta \frac{1}{M} \Delta b^{(l)}$$

步骤5：利用更新后的权值和偏置，重新进行前向传输计算，计算网络输出。

步骤6：重复步骤3、步骤4和步骤5，直至网络输出与期望输出损失函数小于某个设定的阈值或者迭代到某次数，则算法结束。





7.3 BP网络

5、BP网络问题分析

- 神经网络（在训练时，采用误差反向传播(BP)算法）在层次深的情况下性能变得很不理想：传播时容易出现**梯度弥散**，陷入局部最优，且这种情况随着网络层数的增加而更加严重，所以只能处理浅层结构从而限制了性能。

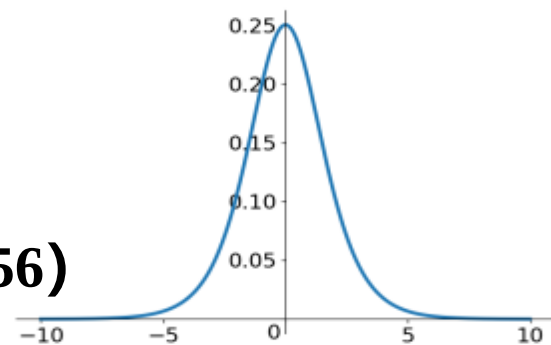
第 l 层 ($2 \leq l \leq L - 1$) 关于权值和偏置的偏导数：

$$\frac{\partial E(i)}{\partial W_{ji}^{(l)}} = \delta_j^{(l)} h_i^{(l-1)} \quad \frac{\partial E(i)}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

误差反向传播

其中， $\delta_j^{(l)} = \sum_{k=1}^{s_{l+1}} W_{kj}^{(l+1)} \delta_k^{(l+1)} f'(x) \Big|_{x = ne_{t_j}^{(l)}}$

激活函数导数最大值为0.25 ($0.25^2=0.0625$; $0.25^3=0.0156$)



Sigmoid激活函数导数



7.4 自组织神经网络

- **感知器和BP神经网络：**有监督学习方式，样本带标签
- **自组织神经网络：**早期无监督学习网络模型的代表，样本不带标签

基本思想：模拟生物神经系统功能的人工神经网络。

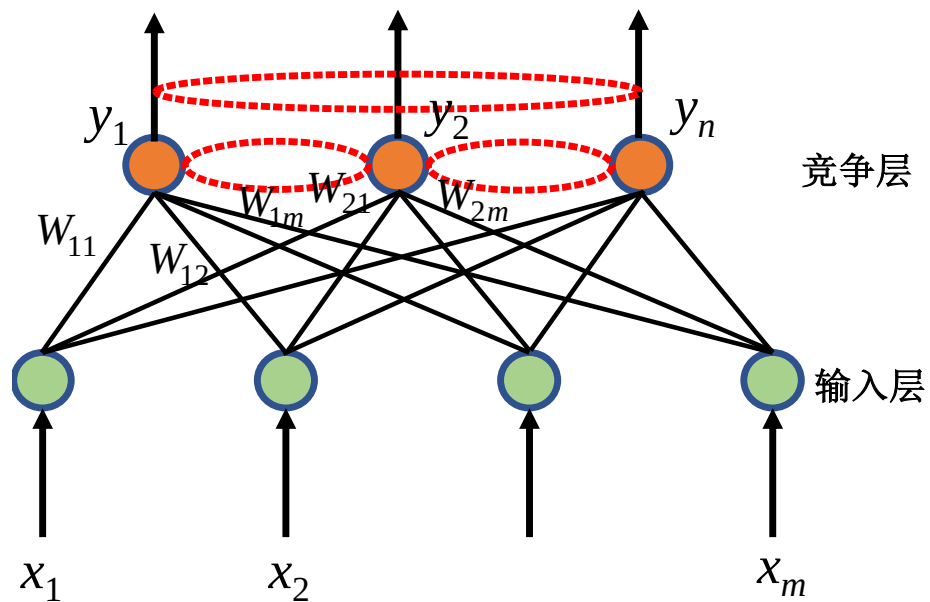
在生物神经系统中，存在着一种**侧抑制现象**，即一个神经细胞兴奋以后，会对周围其他神经细胞产生抑制作用。这种抑制作用会使神经细胞之间出现**竞争**，竞争**获胜的神经细胞兴奋**，**失败的神经细胞被抑制**。



7.4 自组织神经网络

1、自组织竞争神经网络

一种基于竞争性学习的网络模型，输出神经元之间相互竞争激活，在任意时刻**只有一个神经元被激活**。被激活的神经元被称为**胜利者神经元**（winner-takes-all neuron），而其它神经元的状态被抑制，故称为Winner Take All（简称WTA）。



竞争层：对输入模型进行分析比较，寻找规律，并进行归类处理。

输入层：接收外界信息，将输入模式向竞争层传递。



7.4 自组织神经网络

1、自组织竞争神经网络

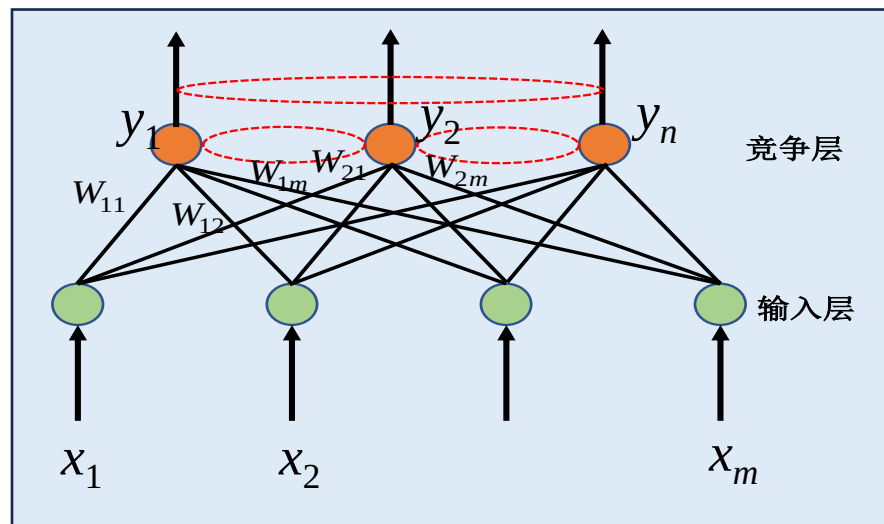
➤ WTA竞争学习过程

步骤1：向量归一化

对网络当前输入向量 X （ m 维列向量）和竞争层中各神经元对应的权重向量 W_j （对应竞争层第 j 神经元，为 m 维列向量 $W_{j1}, W_{j2}, \dots, W_{jm}$ ）全部进行归一化，使得 X 和 W_j 的模为1，得到：

$$\hat{X} = \frac{X}{\|X\|}$$

$$\hat{W}_j = \frac{W_j}{\|W_j\|}$$





7.4 自组织神经网络

1、自组织竞争神经网络

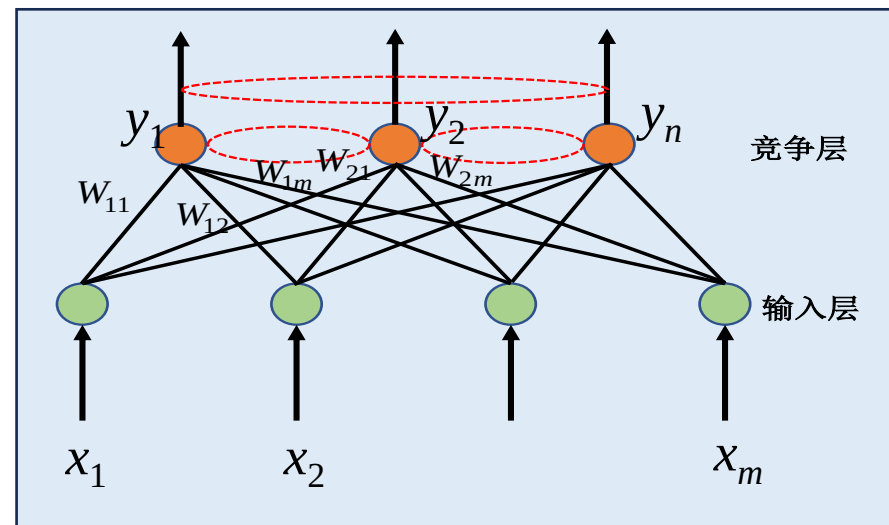
➤ WTA竞争学习过程

步骤2：寻找获胜神经元

将 $\hat{X}$ 与竞争层所有神经元对应的权向量 $\widehat{W}_j (j = 1, 2, \dots, n)$ 进行相似性比较。将最相似的神经元判为竞争获胜神经元：

$$\|\hat{X} - \widehat{W}_{j^*}\| = \min_{j \in \{1, 2, \dots, n\}} \{\|\hat{X} - \widehat{W}_j\|\}$$

经过归一化后，相似度最大可认为是在单位圆
(2D情况) 中找到夹角最小的点。





7.4 自组织神经网络

1、自组织竞争神经网络

➤ WTA竞争学习过程

步骤3：网络输出与权值调整

按WTA学习规则，获胜神经元输出为1，其他神经元输出为0：

$$y_j(t+1) = \begin{cases} 1 & j = j^* \\ 0 & j \neq j^* \end{cases}$$

只有获胜神经元的权值才能进行调整更新：

$$\begin{cases} W_{j^*}(t+1) = \widehat{W}_{j^*}(t) + \Delta W_{j^*} = \widehat{W}_{j^*}(t) + \eta(\widehat{X} - \widehat{W}_{j^*}) \\ W_j(t+1) = \widehat{W}_j(t) & j \neq j^* \end{cases}$$

其中学习率 $0 < \eta \leq 1$ ，一般随着学习的进展而减小，即调整的程度越来越小，神经元（权重）趋于聚类中心。



7.4 自组织神经网络

1、自组织竞争神经网络

➤ WTA竞争学习过程

步骤4：重新归一化处理

归一化后的权向量经过调整后，得到的新权向量 W_j^* 不再是单位向量，因此需要对学习调整后的权向量 W_j^* 重新进行归一化。

$$\widehat{W}_j^* = \frac{W_j^*}{\|W_j^*\|}$$

重复前面的步骤，直到达到指定的迭代次数。



7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

利用竞争学习算法将下列模式分为两类：

$$X_1 = [0.8 \quad 0.6]^T$$

$$X_2 = [0.1736 \quad -0.9848]^T$$

$$X_3 = [0.707 \quad 0.707]^T$$

$$X_4 = [0.342 \quad -0.9397]^T$$

$$X_5 = [0.6 \quad 0.8]^T$$

其中学习率： $\eta = 0.5$



7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

【求解】

将上述输入量转换为极坐标形式，

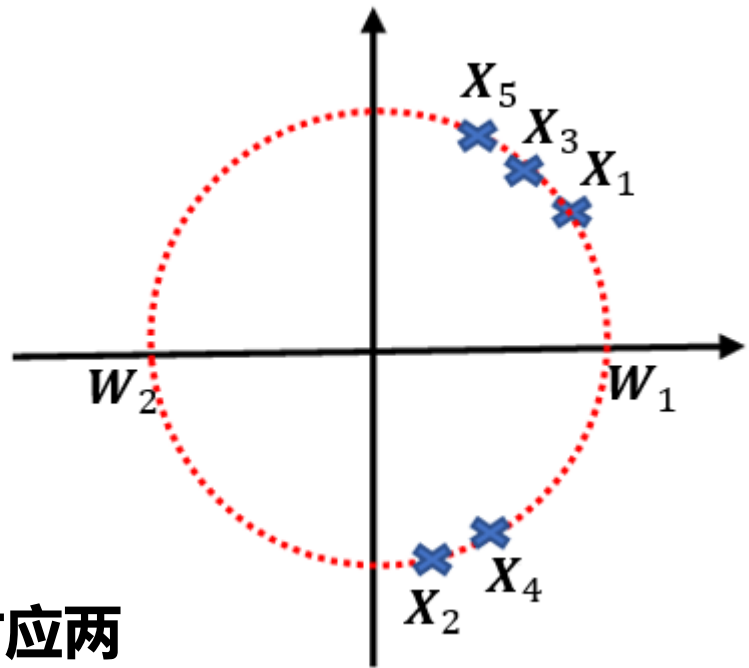
$$X_1 = 1\angle 36.89^\circ, X_2 = 1\angle -80^\circ, X_3 = 1\angle 45^\circ,$$

$$X_4 = 1\angle -70^\circ, X_5 = 1\angle 53.13^\circ$$

要求将如上模式分为两类，则竞争层为两个神经元（对应两个权向量）。两个权向量被随机初始化为单位向量：

$$W_1(0) = [1 \ 0]^T = 1\angle 0^\circ$$

$$W_2(0) = [-1 \ 0]^T = 1\angle -180^\circ$$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

步骤1：对输入 X_1 ，与两个神经元所对应的权向量 $W_1(0)$ 和 $W_2(0)$ 进行相似性比较：

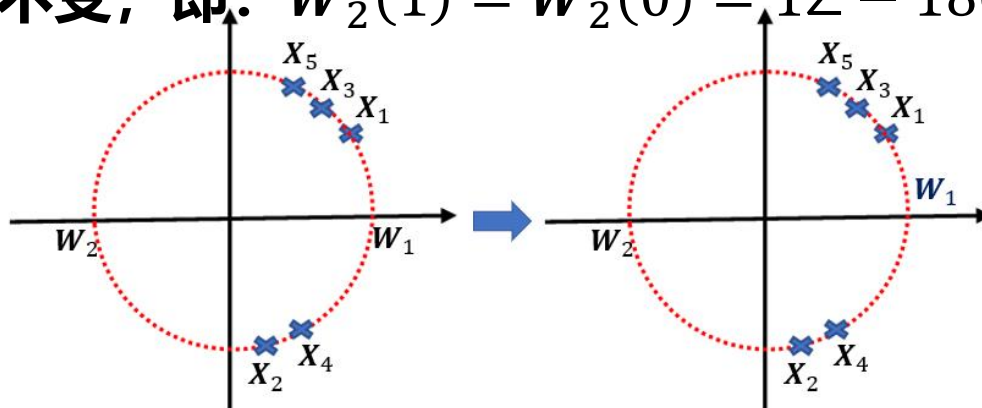
$$d_1 = \|X_1 - W_1(0)\| = 1 \angle 36.89^\circ$$

$$d_2 = \|X_1 - W_2(0)\| = 1 \angle 216.89^\circ$$

因为 $d_1 < d_2$ ，神经元1为获胜神经元，对其权值 W_1 进行调整：

$$W_1(1) = W_1(0) + \eta(X_1 - W_1(0)) = 0 + 0.5(36.89 - 0) = 1 \angle 18.45^\circ$$

神经元2对应的权值 W_2 不变，即： $W_2(1) = W_2(0) = 1 \angle -180^\circ$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

步骤2：对输入 X_2 ，与两个神经元所对应的权向量 $W_1(1)$ 和 $W_2(1)$ 进行相似性比较：

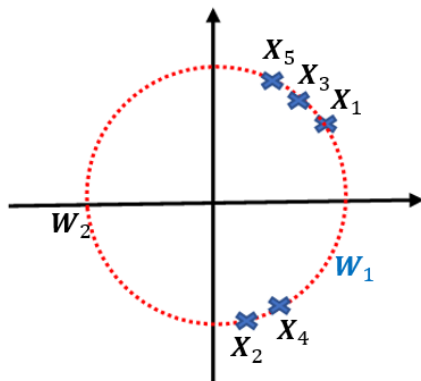
$$d_1 = \|X_2 - W_1(1)\| = 1 \angle 98.43^\circ$$

$$d_2 = \|X_2 - W_2(1)\| = 1 \angle 100^\circ$$

因为 $d_1 < d_2$ ，神经元1为获胜神经元，对其权值 W_1 进行调整：

$$W_1(2) = W_1(1) + \eta(X_2 - W_1(1)) = 18.45 + 0.5(-80 - 18.45) = 1 \angle -30.8^\circ$$

神经元2对应的权值 W_2 不变，即： $W_2(2) = W_2(1) = 1 \angle -180^\circ$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

步骤3：对输入 X_3 ，与两个神经元所对应的权向量 $W_1(2)$ 和 $W_2(2)$ 进行相似性比较：

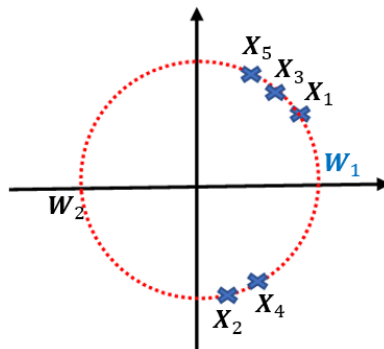
$$d_1 = \|X_3 - W_1(2)\| = 1 \angle 75.8^\circ$$

$$d_2 = \|X_3 - W_2(2)\| = 1 \angle 225^\circ$$

因为 $d_1 < d_2$ ，神经元1为获胜神经元，对其权值 W_1 进行调整：

$$W_1(3) = W_1(2) + \eta(X_3 - W_1(2)) = -30.8 + 0.5(45 + 30.8) = 1 \angle 7^\circ$$

神经元2对应的权值 W_2 不变，即： $W_2(3) = W_2(2) = 1 \angle -180^\circ$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

步骤4：对输入 X_4 ，与两个神经元所对应的权向量 $W_1(3)$ 和 $W_2(3)$ 进行相似性比较：

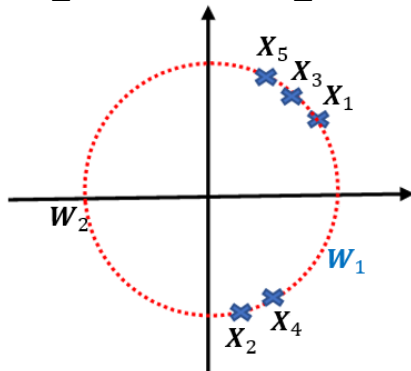
$$d_1 = \|X_4 - W_1(3)\| = 1\angle 77^\circ$$

$$d_2 = \|X_4 - W_2(3)\| = 1\angle 110^\circ$$

因为 $d_1 < d_2$ ，神经元1为获胜神经元，对其权值 W_1 进行调整：

$$W_1(4) = W_1(3) + \eta(X_4 - W_1(3)) = 7 + 0.5(-70 - 7) = 1\angle -31.5^\circ$$

神经元2对应的权值 W_2 不变，即： $W_2(4) = W_2(3) = 1\angle -180^\circ$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

步骤5：对输入 X_5 ，与两个神经元所对应的权向量 $W_1(4)$ 和 $W_2(4)$ 进行相似性比较：

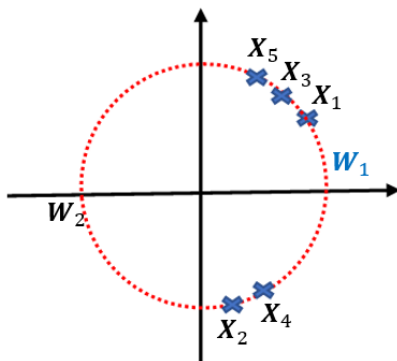
$$d_1 = \|X_5 - W_1(4)\| = 1 \angle 84.63^\circ$$

$$d_2 = \|X_5 - W_2(4)\| = 1 \angle 126.87^\circ$$

因为 $d_1 < d_2$ ，神经元1为获胜神经元，对其权值 W_1 进行调整：

$$W_1(5) = W_1(4) + \eta(X_5 - W_1(4)) = -31.5 + 0.5(53.13 + 31.5) \approx 1 \angle 11^\circ$$

神经元2对应的权值 W_2 不变，即： $W_2(5) = W_2(4) = 1 \angle -180^\circ$





7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析

至此形成一个循环。重复循环直至收敛或迭代至特定循环次数。

学习次数	W_1	W_2	学习次数	W_1	W_2
1	18.45°	-180°	11	40.5°	-100°
2	-30.8°	-180°	12	40.5°	-90°
3	7°	-180°	13	43°	-90°
4	-32°	-180°	14	43°	-81°
5	11°	-180°	15	47.5°	-81°
6	24°	-180°	16	42°	-81°
7	24°	-130°	17	42°	-80.5°
8	34°	-130°	18	43.5°	-80.5°
9	34°	-100°	19	43.5°	-75°
10	44°	-100°	20	48.5°	-75°

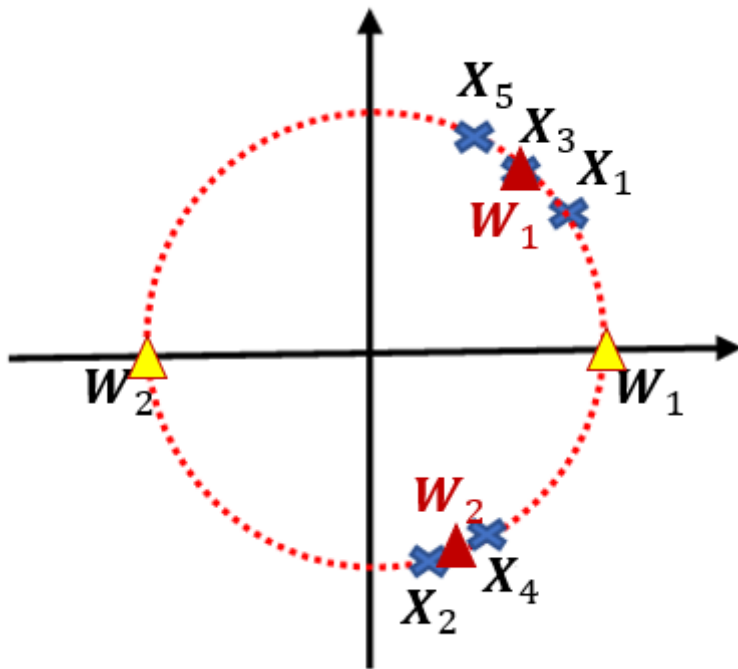
W_1 和 W_2 逐渐稳定于两类聚类中心 $\frac{1}{3}(X_1 + X_3 + X_5) = 1\angle 45^\circ$ $\frac{1}{2}(X_2 + X_4) = 1\angle -75^\circ$



7.4 自组织神经网络

1、自组织竞争神经网络

➤ 算例分析



首先给定训练数据（蓝色叉号）和神经元权重初始值（黄色三角）

迭代训练之后，神经元权重趋向于聚类中心（红色三角）；

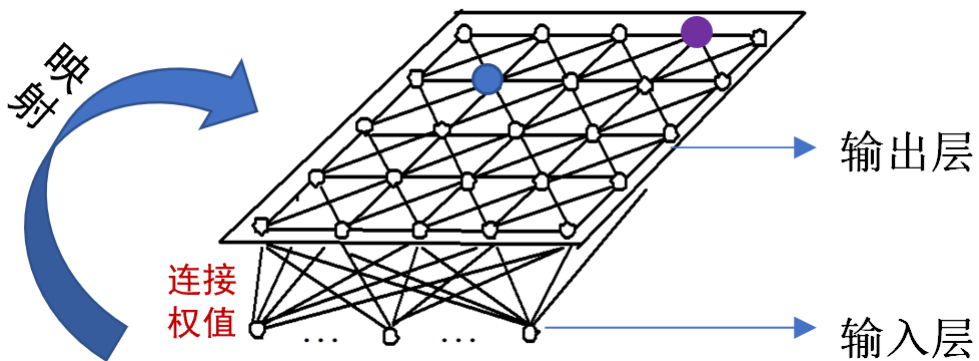
Test阶段，给定数据点，基于WTA策略，直接算出和哪个神经元权重最相似就分到哪个类。



7.4 自组织神经网络

2、自组织映射网络

- Self-Organizing Map, 简称SOM网。
- 一种竞争学习型的无监督神经网络。
- 能将高维度输入数据映射到低维空间（通常为二维），同时保持输入数据在高维空间的拓扑结构，即将高维空间中相似的样本点映射到网络输出层中的邻近神经元。



输出：二维拓扑结构

神经元以矩阵方式排列在二维空间中，每个神经元都有一个权值向量

输入向量 x_1 (高维度)

输入向量 x_2 (高维度)

.....

假设一个输入样本为 x_i 是一个 m 维向量，则输入层神经元个数为 m 个。



7.4 自组织神经网络

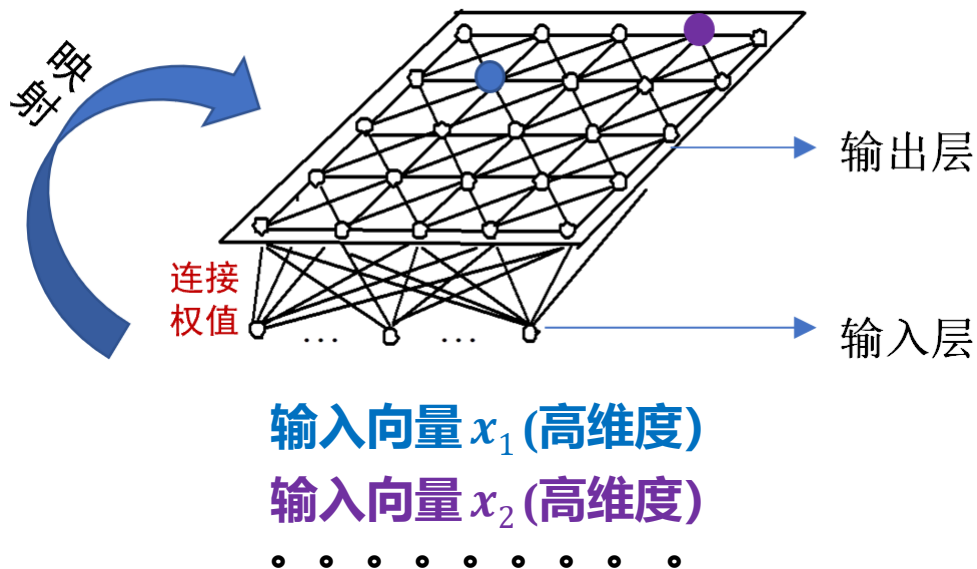
2、自组织映射网络

- **基本思想：**通过学习输入空间中的数据，生成一个低维、离散的**映射**(Map)，从某种程度上也可看成一种**降维算法**。
- **无监督人工神经网络：**不同于一般神经网络基于损失函数的反向传递来训练，而是运用**竞争学习**策略，依靠神经元之间互相竞争逐步优化网络。利用**近邻关系函数**来**维持输入空间的拓扑结构**。
- **维持输入空间的拓扑结构：**二维映射包含了数据点之间的相对距离。输入空间中相邻的样本会被映射到相邻的输出神经元。
- 无需样本标签，**可用于聚类**。



7.4 自组织神经网络

2、自组织映射网络



(神经元以矩阵方式排列在二维空间中)

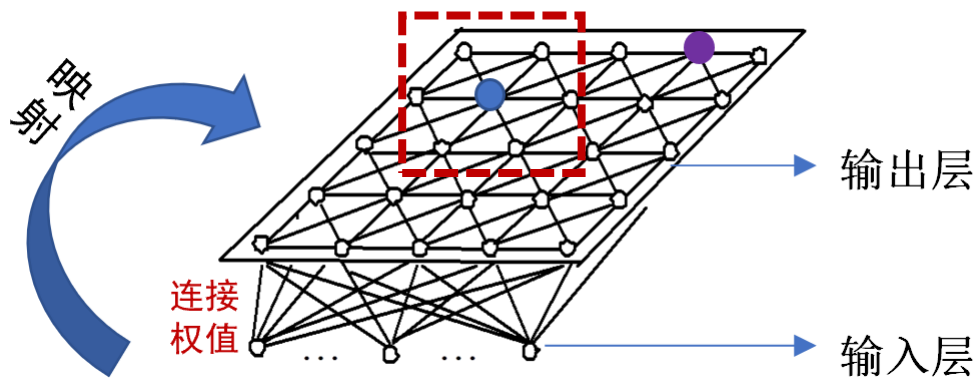
➤ SOM训练目标

为每个输出层神经元找到合适的权向量，达到保持拓扑结构的目的。



7.4 自组织神经网络

2、自组织映射网络



(神经元以矩阵方式排列在二维空间中)

输入向量 x_1 (高维度)

输入向量 x_2 (高维度)

• • • • •

➤ SOM训练过程

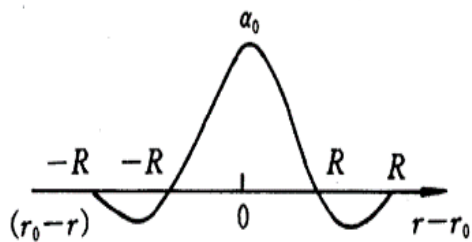
- 接收到一个训练样本后，计算输出层每个神经元的权向量与该输入样本之间的距离，距离最近的神经元成为竞争获胜者（最佳匹配单元）。
- 最佳匹配单元及其近邻神经元的权向量将被调整，以使得这些权向量与当前输入样本的距离缩小；
- 该过程不断迭代，直至收敛。



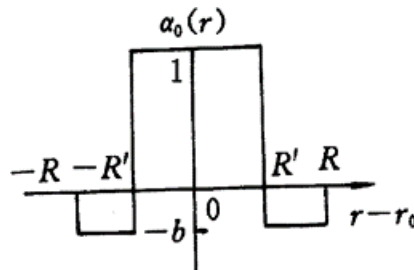
7.4 自组织神经网络

2、自组织映射网络

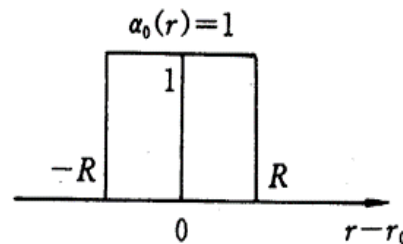
➤ 常见调整函数



墨西哥草帽函数



大礼帽函数



厨师帽函数

- 以**获胜神经元**为中心设定一个**邻域半径 R** ，该半径圈定的范围称为**优胜邻域**。优胜邻域内的所有神经元均会进行权值调整，且权值调整的程度与其距离获胜神经元的距离相关。
- 优胜邻域开始被设为较大范围，随着训练次数的增加范围不断收缩，最终收缩到半径为零。



7.4 自组织神经网络

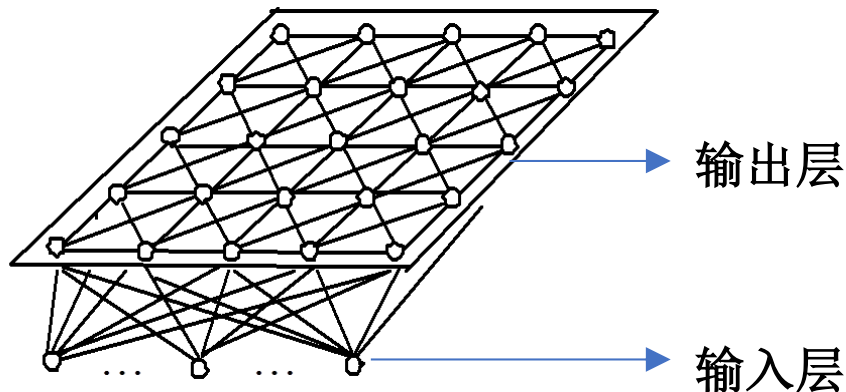
2、自组织映射网络

➤ SOM算法学习步骤

步骤1： 确定SOM网络拓扑结构，根据样本类型和经验确定网络竞争层维度、神经元数量和神经元的拓扑结构

a) 设定输出层神经元的数量：

输出层神经元的数量和训练集样本的类别数相关。 若不清楚类别数，尽可能地设定较多的节点数，以便较好地映射样本的拓扑结构，如果分类过细再适当减少输出节点。





7.4 自组织神经网络

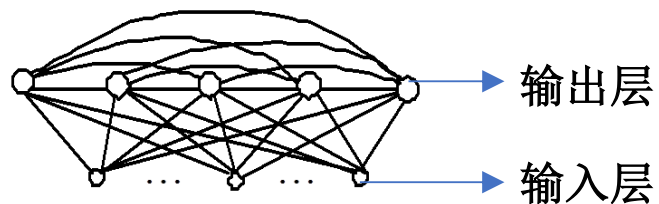
2、自组织映射网络

➤ SOM算法学习步骤

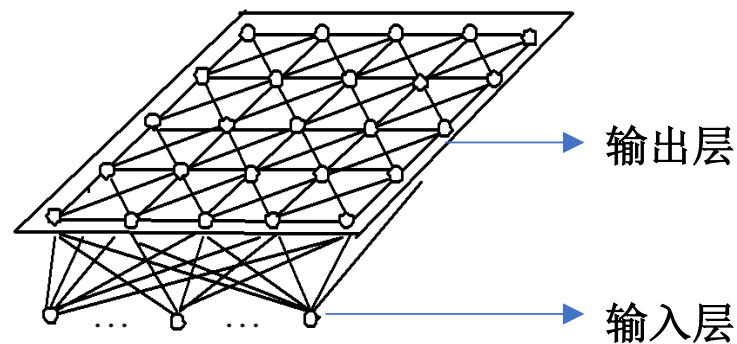
步骤1： 确定SOM网络拓扑结构，根据样本类型和经验确定网络竞争层维度、神经元数量和神经元的拓扑结构

b) 设计输出层节点的排列：

与实际应用需求相关。 一般根据预估的聚类簇数进行设置。比如要分成4类，那可以将输出层（竞争层）设计为 2×2 ， 1×4 ， 4×1 。



一维线阵



二维平面阵



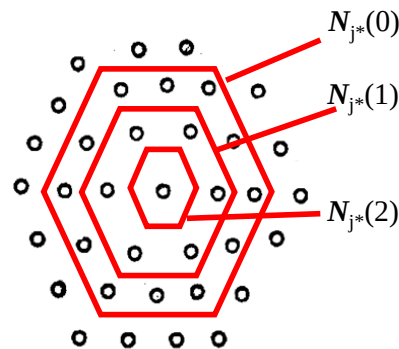
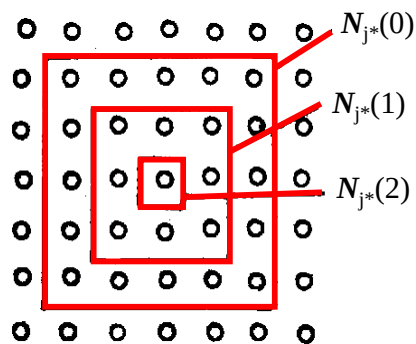
7.4 自组织神经网络

2、自组织映射网络

➤ SOM算法学习步骤

步骤2：初始化

- a) 对竞争层（也是输出层）各神经元的权值赋小的随机数初值，该操作可使权向量充分分散在样本空间中。对权值进行归一化处理，得到 $\widehat{W}_j$ ($j = 1, 2, \dots, n$, n 为输出层神经元数量)
- b) 设定初始优胜邻域 $N_{j^*}(0)$ ，邻域形状可以是正方形、六边形等。
- c) 设定学习率初始值 η ，在训练开始时，学习率可以选取较大的值，之后以较快的速度下降





7.4 自组织神经网络

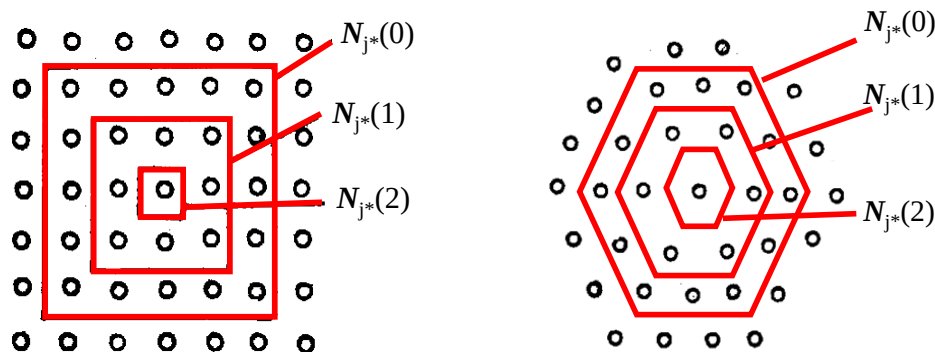
2、自组织映射网络

➤ SOM算法学习步骤

步骤3：从训练数据中随机选取一个输入数据，并对其进行归一化，得到 $\hat{x}$

步骤4：寻找获胜神经元。计算 $\hat{x}$ 与每个神经元归一化权值 $\hat{w}_j$ 内积，找到内积最大的获胜神经元 j^*

步骤5：定义优胜邻域 $N_{j^*}(t)$ 。以 j^* 为中心确定 t 时刻的权值调整域，一般选择初始邻域 $N_{j^*}(0)$ 较大，训练时 $N_{j^*}(t)$ 随训练时间逐渐收缩。





7.4 自组织神经网络

2、自组织映射网络

➤ SOM算法学习步骤

步骤6：调整权重。对优胜邻域 $N_{j^*}(t)$ 内的所有神经元进行权重调整

$$W_{ij}(t+1) = W_{ij}(t) + \eta(t, N)[x_i^p - W_{ij}(t)] \quad i = 1, 2, \dots, m, j \in N_{j^*}(t)$$

下标 i ：输入层神经元索引； m 为输入层神经元总数量

下标 j^* ：输出层获胜神经元索引；

下标 j ：获胜神经元优胜邻域内的神经元索引； $j \in N_{j^*}(t)$

t ：训练时间或者训练次数；

N ：获胜神经元 j^* 邻域第 j 个神经元与获胜神经元 j^* 之间的拓扑距离

$\eta(t, N)$ ：为训练时间为 t 、邻域内第 j 个神经元与获胜神经元 j^* 之间拓扑距离为 N 情况下的学习率函数。

$$\eta(t, N) = \eta(t)e^{-N} \rightarrow \eta(t): t \text{ 的单调递减函数}$$



7.4 自组织神经网络

2、自组织映射网络

➤ SOM算法学习步骤

步骤6：调整权重。对优胜邻域 $N_{j^*}(t)$ 内的所有神经元进行权重调整

$$W_{ij}(t+1) = W_{ij}(t) + \eta(t, N)[x_i^p - W_{ij}(t)] \quad i = 1, 2, \dots, m, j \in N_{j^*}(t)$$

$\eta(t, N)$ ：为训练时间为 t 、邻域内第 j 个神经元与获胜神经元 j^* 之间拓扑距离为 N 情况下的学习率函数。

$$\eta(t, N) = \eta(t)e^{-N} \rightarrow \eta(t) : t \text{ 的单调递减函数}$$



在训练开始时，学习率可以选取较大的值，之后以较快的速度下降，这样有利于很快地捕捉到输入向量的大致结构，然后学习率在较小的值上缓降至0值，可以精细地调整权值，使得符合输入空间的样本分布结构。

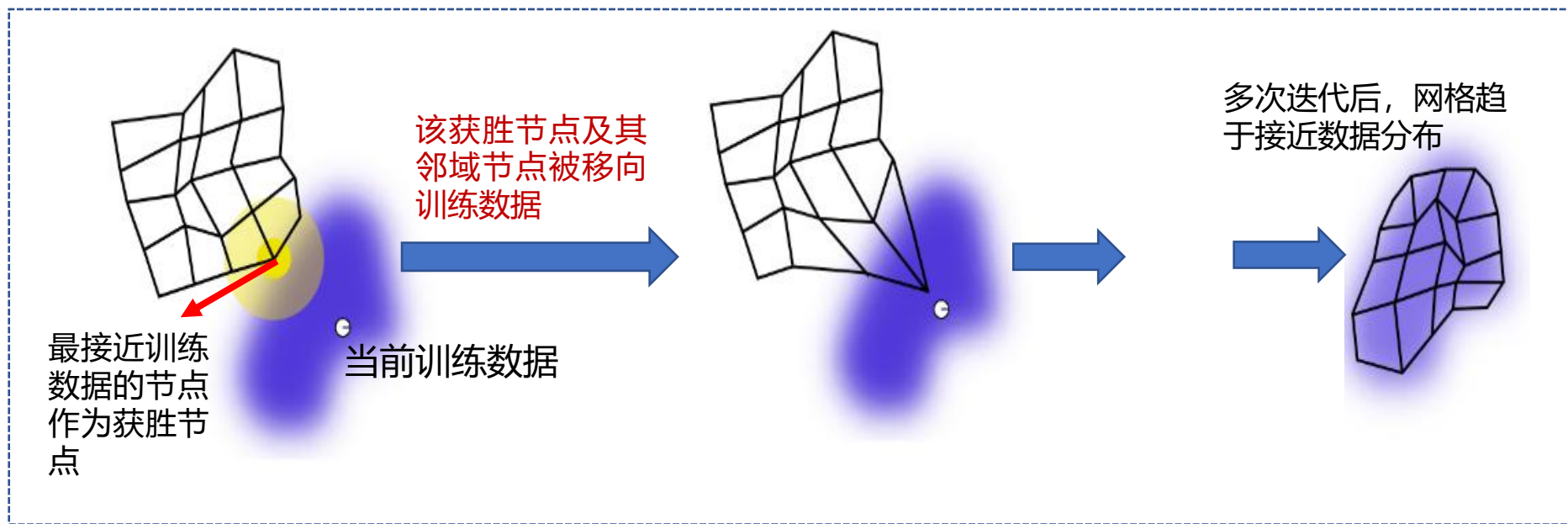


7.4 自组织神经网络

2、自组织映射网络

➤ SOM算法学习步骤

步骤7：结束判定。查看学习率是否减小到0，或者小于某个阈值。不满足条件时，转至**步骤3**，继续训练。



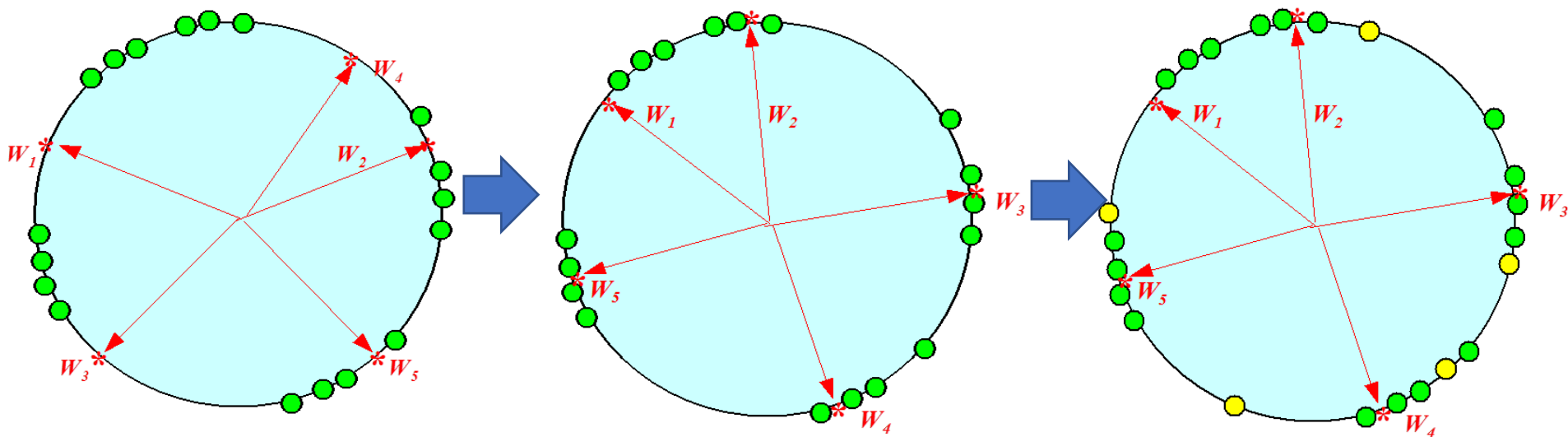
SOM 网络学习过程示意图



7.4 自组织神经网络

2、自组织映射网络

➤ SOM算法工作原理示意图



首先给定训练数据
(绿点) 和神经元权重
初始值 (红点)

迭代训练之后,
神经元权重趋向
于聚类中心;

Test阶段, 给定数据点 (黄点), 基于WTA策略, 用内积直接算出和哪个神经元权值最相似就分到哪个类。



7.4 自组织神经网络

2、自组织映射网络

➤ SOM网络与K均值对比

- a) **K均值算法为每个输入数据找到一个最相似的类后，只更新这个类的参数（重新计算该类的聚类中心）；自组织映射神经网络则会更新邻近节点。**
- b) **K均值算法受噪声影响较大，而自组织映射神经网络的准确性可能会比 K均值算法低（因为也更新了邻近节点，噪声点所对应的神经元节点同样被更新，加大了噪声干扰）**



7.4 自组织神经网络

3、小结

➤ 自组织神经网络特点

- a) 神经网络，竞争学习策略；
- b) 无监督学习，不需要额外标签；
- c) 非常适合高维数据的可视化，能够维持输入空间的拓扑结构；

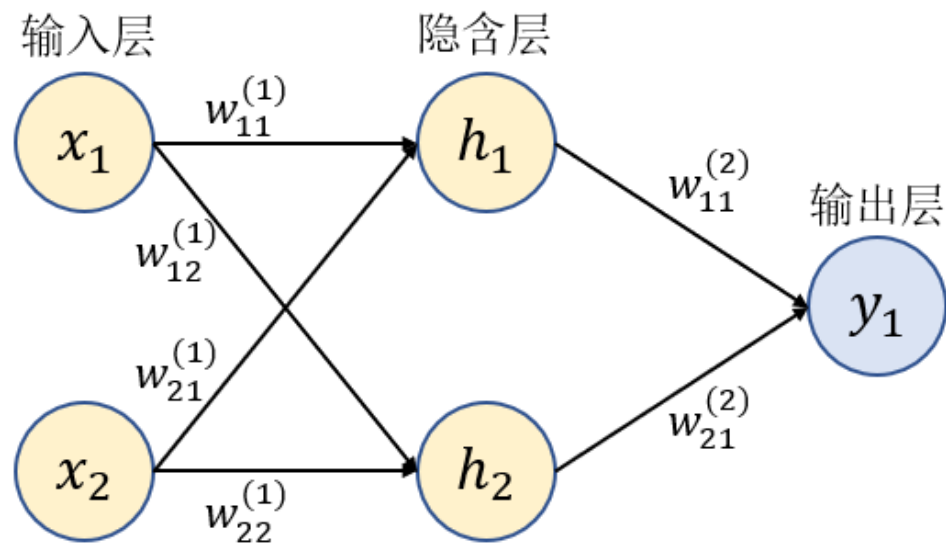


作业

1、对如下BP神经网络，设定学习率 $\eta = 1$ ，各点偏置量均为0，激活函数采用Sigmoid函数。输入样本 $x_1 = 1, x_2 = 0$ ，输出节点 y 的期望输出为1。第 k 次学习得到的权值分别为

$$w_{11}^{(1)}(k) = 0 \quad w_{12}^{(1)}(k) = 2 \quad w_{21}^{(1)}(k) = 1 \quad w_{22}^{(1)}(k) = 1$$

$$w_{11}^{(2)}(k) = 1 \quad w_{21}^{(2)}(k) = 1$$





作业

【求解】

a) 第 k 次学习得到节点输出 $y(k)$

b) 第 $k + 1$ 次学习得到的各点权值

$$w_{11}^{(1)}(k + 1) \quad w_{12}^{(1)}(k + 1) \quad w_{21}^{(1)}(k + 1) \quad w_{22}^{(1)}(k + 1)$$

$$w_{11}^{(2)}(k + 1) \quad w_{21}^{(2)}(k + 1)$$

c) 第 $k + 1$ 次学习得到节点输出 $y(k + 1)$



北京航空航天大学
BEIHANG UNIVERSITY

德才兼备 知行合一

The end